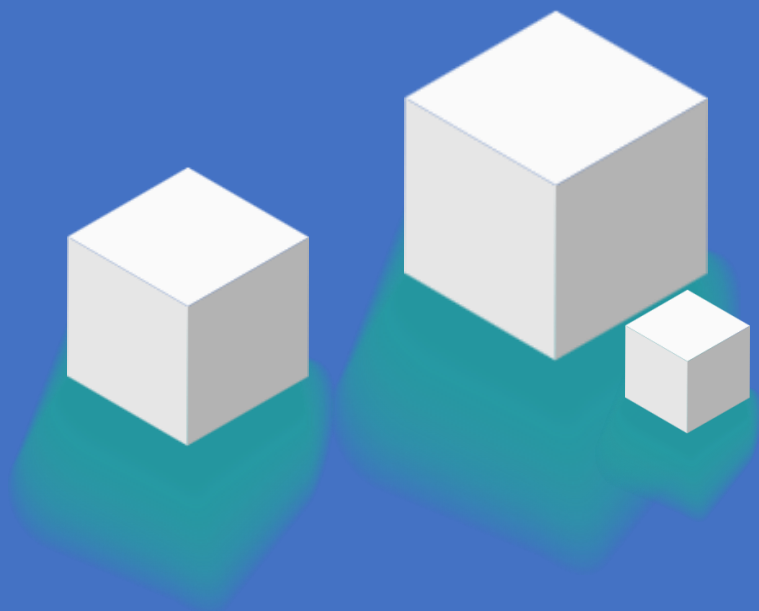


项目实战：交互式BI报表



>> 今天的学习目标

项目实战：交互式BI报表

- ChatBI的原理
- 数据采集
- 可视化组件
- 查询历史价格
- 对比分析
- 热点新闻
- 智能分析
 - 趋势预测（ARIMA分析）
 - 异常检测（Boll带分析）
 - 周期性分析（prophet）

ChatBI原理

ChatBI 的核心原理：通过Text-to-SQL，将自然语言查询（如“上季度销售额最高的产品”）自动转换为可执行的SQL语句，从而让非技术用户直接与数据库交互。其关键技术包括：

- **语义解析**：利用大模型（如Qwen3）理解用户意图，识别查询中的实体（表、字段）和操作（排序、聚合）。
 - **SQL生成**：基于大模型（如Qwen3）将语义结构转化为合规SQL，需解决模式链接（schema linking）问题，即匹配用户术语与数据库实际表名/字段名。
 - **交互优化**：支持模糊查询（如“卖得不好的产品”需定义“不好”的阈值）和纠错（如表名歧义时询问用户）。
- 数据库结构复杂时准确率下降，需结合Few-shot学习或微调；
 - 动态数据（如实时更新）可能影响结果一致性。
 - ChatBI大幅降低数据分析门槛，但依赖高质量的数据字典和持续训练优化。

项目实战：交互式BI报表

TO DO：搭建交互式BI，一个集数据采集、存储、分析和可视化于一体的交互式商业智能报表平台，以金融证券数据分析为例。

系统通过采集贵州茅台、五粮液、国泰君安、中芯国际等历史价格，结合多种分析模型，为用户提供多维度的数据洞察：

- **交互式分析**：用户可通过自然语言查询获取柱状图、折线图等可视化结果，快速掌握股票走势、成交量等关键指标。
- **实时新闻查询**：通过自然语言，了解股票当前的热点新闻
- **智能预测模型**：

ARIMA趋势预测：为企业投资决策提供未来价格走势参考

Boll带异常检测：识别超买超卖点，辅助风险控制

Prophet周期性分析：发现潜在季节性规律，优化交易策略

数据采集

帮我编写Python，使用tushare，获取贵州茅台、五粮液、国泰君安、中芯国际的历史价格，从2020-01-01到今天，保存到.xlsx中

我会帮你创建一个Python脚本来获取这些股票的历史数据。首先，我们需要创建一个新的Python文件来实现这个功能。

让我先创建一个新的Python文件：

```
stock_data.py (new) +59 •
import tushare as ts
import pandas as pd
from datetime import datetime
import os

# 设置tushare token
# 注意：使用前需要先注册tushare账号并获取token
ts.set_token('your_token_here') # 请替换为你
pro = ts.pro_api()

# 定义股票代码列表
stock_codes = {
    '贵州茅台': '600519.SH',
    '五粮液': '000858.SZ',
```



```
stock_data.py > ...
1 import tushare as ts
2 import pandas as pd
3 from datetime import datetime
4 import os
5
6 # 设置tushare token
7 # 注意：使用前需要先注册tushare账号并获取token
8 ts.set_token('your_token_here') # 请替换为你的token
9 pro = ts.pro_api()
10
11 # 定义股票代码列表
12 stock_codes = {
13     '贵州茅台': '600519.SH',
14     '五粮液': '000858.SZ',
15     '国泰君安': '601211.SH',
16     '中芯国际': '688981.SH'
17 }
18
19 # 获取当前日期
20 end_date = datetime.now().strftime('%Y%m%d')
21
22 # 创建DataFrame列表用于存储所有股票数据
23 all_stocks_data = []
24
25 # 获取每只股票的数据
26 for stock_name, stock_code in stock_codes.items():
27     try:
28         # 获取日线数据
29         df = pro.daily(ts_code=stock_code,
30                        start_date='20200101',
31                        end_date=end_date)
```

这里需要替换你的api key

数据采集 (Tushare使用)

Tushare 是一个免费、开源的 Python 财经数据接口库，主要用于获取国内金融市场的数据。

- 数据丰富：涵盖股票、指数、基金、期货、期权、外汇、宏观经济等多种金融数据。
- 免费可用：基础数据免费，部分高频或深度数据需注册或付费。
- 接口简单：通过 Python 调用，返回 Pandas 的 DataFrame 格式，方便数据处理。
- 支持多种数据源：整合了交易所、财经网站等公开数据。



<http://tushare.pro/user/token>

数据采集

stock_name	ts_code	trade_date	open	high	low	close	vol	amount
贵州茅台	600519.SH	2025-06-17	1420	1427	1412.18	1427	27231.51	3868365
贵州茅台	600519.SH	2025-06-16	1401.2	1425.04	1401.18	1422.29	42887.87	6060448
贵州茅台	600519.SH	2025-06-13	1444.41	1449.97	1425.06	1426.95	59113.25	8466516
贵州茅台	600519.SH	2025-06-12	1480	1483.87	1454.1	1459	49903.61	7306801
贵州茅台	600519.SH	2025-06-11	1476.8	1494	1476.1	1480	30892.82	4585523
贵州茅台	600519.SH	2025-06-10	1486	1492	1474.8	1475.01	30492.2	4511935
贵州茅台	600519.SH	2025-06-09	1505.02	1508.7	1485.68	1486.11	44958.67	6723839
贵州茅台	600519.SH	2025-06-06	1513.8	1518.97	1503	1506.39	25023.07	3777728
贵州茅台	600519.SH	2025-06-05	1514	1516	1502.2	1514	23213.66	3504926
贵州茅台	600519.SH	2025-06-04	1510.5	1523	1509.22	1509.96	23008.09	3480364
贵州茅台	600519.SH	2025-06-03	1508.01	1519	1505.02	1509	28979.18	4377893
贵州茅台	600519.SH	2025-05-30	1540.15	1545	1515.24	1522	31239.18	4764876
贵州茅台	600519.SH	2025-05-29	1540	1555	1532	1540	21859.58	3375466
贵州茅台	600519.SH	2025-05-28	1543.99	1546.46	1533	1537	16265.68	2504625
贵州茅台	600519.SH	2025-05-27	1551	1558.6	1543.51	1543.9	17752.13	2748946
贵州茅台	600519.SH	2025-05-26	1570	1574.63	1545	1550.5	27086.53	4212944
贵州茅台	600519.SH	2025-05-23	1575.2	1587.9	1571.66	1572.6	21537	3399425
贵州茅台	600519.SH	2025-05-22	1580.99	1584.99	1570.1	1580	15090.24	2381237
贵州茅台	600519.SH	2025-05-21	1585	1595.94	1580.97	1580.97	19794.64	3143941
贵州茅台	600519.SH	2025-05-20	1580	1594.8	1575.1	1586	19521.53	3095743
贵州茅台	600519.SH	2025-05-19	1596	1600	1573.88	1578.98	38062.83	6021950
贵州茅台	600519.SH	2025-05-16	1633.99	1636.99	1614.13	1614.13	22935.22	3714509
贵州茅台	600519.SH	2025-05-15	1634.8	1643.59	1624.13	1632.01	24732.85	4043403
贵州茅台	600519.SH	2025-05-14	1590	1645	1588.18	1634.99	39460.12	6394735
贵州茅台	600519.SH	2025-05-13	1608.92	1608.92	1585.11	1590.3	21258.29	3386618
贵州茅台	600519.SH	2025-05-12	1598	1618.93	1596.61	1604.5	24735.33	3967786
贵州茅台	600519.SH	2025-05-09	1578.99	1597.45	1575.05	1591.18	23671.9	3757575
贵州茅台	600519.SH	2025-05-08	1553	1592.78	1549.83	1578.19	33481.06	5265446
贵州茅台	600519.SH	2025-05-07	1570	1570	1550.2	1555	27462.21	4275143
贵州茅台	600519.SH	2025-05-06	1559	1559	1544.33	1550.2	18300.26	2838614
贵州茅台	600519.SH	2025-04-30	1549.99	1566.66	1546.3	1547	25754.25	4005439
贵州茅台	600519.SH	2025-04-29	1550	1552.54	1532.02	1544	18921.8	2917423
贵州茅台	600519.SH	2025-04-28	1552	1555	1546.6	1550	14659.72	2274252

运行Python，可以得到 stock_history_data.xlsx

我们可以看到tushare保存的历史价格结构：

stock_name, ts_code, trade_date, open, high, low, close, vol, amount

=> 需要按照 trade_date从小到大排序

=> 生成.sql，创建MySQL数据表

=> 将数据导入到 MySQL数据表中

数据采集

将数据按照 trade_date 从小到大排序

查看生成的 stock_history_data.xlsx 的前5行数据，生成对应的MySQL建表语句写入到 .sql

```
CREATE TABLE stock_history_data (  
    id INT AUTO_INCREMENT PRIMARY KEY COMMENT '自增主键',  
    stock_name VARCHAR(20) NOT NULL COMMENT '股票名称',  
    ts_code VARCHAR(20) NOT NULL COMMENT '股票代码',  
    trade_date DATE NOT NULL COMMENT '交易日期',  
    open DECIMAL(15,2) COMMENT '开盘价',  
    high DECIMAL(15,2) COMMENT '最高价',  
    low DECIMAL(15,2) COMMENT '最低价',  
    close DECIMAL(15,2) COMMENT '收盘价',  
    vol DECIMAL(20,2) COMMENT '成交量',  
    amount DECIMAL(20,2) COMMENT '成交额',  
    UNIQUE KEY uniq_stock_date (ts_code, trade_date)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='股票历史价格数据表';
```


数据采集

将建表语句 stock_history_data.sql 导入到 MySQL中，运行后可以看到响应的数据表

这里可以根据自己的情况，修改数据表名称，比如改成 stock_price

然后将 stock_history_data.xlsx 导入到对应的数据表中

导入向导

我们已收集向导导入数据时所需的全部信息。点击 [\[开始\]](#) 按钮进行导入。

表: 1

已处理: 5,133

错误: 5,133

已添加: 0

已更新: 0

已删除: 0

时间: 00:01.58

[IMP] Import start

[IMP] Import type - Excel file

[IMP] Import from - D:\推荐系统\知乎\24-项目实战：交互式BI报表\CASE-股票数据查询\stock_history_data.xlsx

[IMP] Import data [stock_price]

[ERR] 1292 - Incorrect date value: '0000-00-00' for column 'trade_date' at row 1

[ERR] INSERT INTO `stock`.`stock\_price` (`stock\_name`,`ts\_code`,`trade\_date`,`open`,`high`,`low`,`close`,`vol`,`amount`) VALUES ('贵州茅台','600519.SH','0000-00-00',1128,1145.06,1116,1130,148099.16,16696837.095),('国泰君安','601211.SH','0000-00-00',18.8,18.95,18.6,18.67,579913.83,1088983.756),('五粮液','000858.SZ','0000-00-00',132,133.5,129.59,132.08,306674.39,4038536.854),('国泰君安','601211.SH','0000-00-00',18.66,18.9,18.62,18.82,288806.18,720101.025),('贵州茅台','600519.SH','0000-00-00',1128,1145.06,1116,1130,148099.16,16696837.095)

保存

日志

<<

< 上一步

开始

关闭

导入数据时发生错误，可以看到是因为 trade_date没有解析正确。

这里建议将 trade_date类型调整为 varchar(10)

即用字符串方式替换原有的 DATE 类型

数据采集

删除掉原有的 stock_price数据表，然后用更新的建表语句进行创建，并导入 stock_history_data.xlsx

```
CREATE TABLE stock_price (  
    id INT AUTO_INCREMENT PRIMARY KEY COMMENT '自增主键',  
    stock_name VARCHAR(20) NOT NULL COMMENT '股票名称',  
    ts_code VARCHAR(20) NOT NULL COMMENT '股票代码',  
    trade_date VARCHAR(10) NOT NULL COMMENT '交易日期',  
    open DECIMAL(15,2) COMMENT '开盘价',  
    high DECIMAL(15,2) COMMENT '最高价',  
    low DECIMAL(15,2) COMMENT '最低价',  
    close DECIMAL(15,2) COMMENT '收盘价',  
    vol DECIMAL(20,2) COMMENT '成交量',  
    amount DECIMAL(20,2) COMMENT '成交额',  
    UNIQUE KEY uniq_stock_date (ts_code, trade_date)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='股票历史价格数据表';
```

id	stock_name	ts_code	trade_date	open	high	low	close	vol	amount
1	贵州茅台	600519.SH	2020-01-02	1128.00	1145.06	1116.00	1130.00	148099.16	16696837.10
2	国泰君安	601211.SH	2020-01-02	18.80	18.95	18.60	18.67	579913.83	1088983.76
3	五粮液	000858.SZ	2020-01-02	132.00	133.50	129.59	132.08	306674.39	4038536.85
4	国泰君安	601211.SH	2020-01-03	18.66	18.90	18.62	18.83	388806.18	730191.04
5	贵州茅台	600519.SH	2020-01-03	1117.00	1117.00	1076.90	1078.56	130318.78	14266380.60
6	五粮液	000858.SZ	2020-01-03	131.60	132.07	129.61	130.55	204692.48	2672531.47
7	五粮液	000858.SZ	2020-01-06	130.00	130.25	128.52	129.20	259369.79	3353682.07
8	贵州茅台	600519.SH	2020-01-06	1070.86	1092.90	1067.30	1077.99	63414.78	6853917.60
9	国泰君安	601211.SH	2020-01-06	18.66	19.35	18.63	19.02	608640.13	1159450.08
10	国泰君安	601211.SH	2020-01-07	19.09	19.24	18.86	19.04	323597.14	616014.30
11	五粮液	000858.SZ	2020-01-07	129.50	131.07	129.00	129.37	223277.93	2901574.23
12	贵州茅台	600519.SH	2020-01-07	1077.50	1099.00	1076.40	1094.53	47853.59	5220697.07
13	贵州茅台	600519.SH	2020-01-08	1085.05	1095.50	1082.58	1088.14	25008.25	2720371.92
14	国泰君安	601211.SH	2020-01-08	18.80	18.87	18.57	18.63	399607.21	747814.32
15	五粮液	000858.SZ	2020-01-08	128.99	129.76	128.05	128.89	161802.18	2083242.87
16	五粮液	000858.SZ	2020-01-09	129.00	131.20	128.25	130.77	255490.57	3317934.21
17	贵州茅台	600519.SH	2020-01-09	1094.00	1105.39	1090.00	1102.70	37405.87	4110949.81
18	国泰君安	601211.SH	2020-01-09	18.89	18.99	18.76	18.85	266967.08	503807.40
19	贵州茅台	600519.SH	2020-01-10	1109.00	1115.99	1102.50	1112.50	35975.87	3993457.12
20	五粮液	000858.SZ	2020-01-10	130.50	134.05	130.50	133.62	429695.42	5712380.86
21	国泰君安	601211.SH	2020-01-10	19.08	19.21	18.78	18.90	284312.01	540854.17
22	贵州茅台	600519.SH	2020-01-13	1112.50	1129.20	1112.00	1124.27	38515.75	4325687.96
23	国泰君安	601211.SH	2020-01-13	18.85	19.08	18.61	19.08	268102.64	505320.47
24	五粮液	000858.SZ	2020-01-13	134.26	140.00	134.25	139.66	496041.94	6847185.99
25	贵州茅台	600519.SH	2020-01-14	1124.20	1124.89	1103.00	1107.40	35144.65	3916300.79
26	国泰君安	601211.SH	2020-01-14	19.10	19.18	18.87	18.91	246766.96	469716.53
27	五粮液	000858.SZ	2020-01-14	140.10	140.68	138.20	138.56	199073.17	2767444.88
28	贵州茅台	600519.SH	2020-01-15	1109.01	1121.60	1105.00	1112.13	26029.11	2895603.67
29	国泰君安	601211.SH	2020-01-15	18.86	18.95	18.70	18.71	162245.15	304774.49
30	五粮液	000858.SZ	2020-01-15	138.56	142.81	138.55	140.80	234789.04	3309331.11
31	五粮液	000858.SZ	2020-01-16	140.80	142.00	139.30	140.66	145359.88	2046672.62
32	贵州茅台	600519.SH	2020-01-16	1118.87	1118.87	1102.58	1107.00	23191.65	2571593.14
33	国泰君安	601211.SH	2020-01-16	18.80	18.81	18.56	18.62	236394.23	440908.27

查询历史价格

TO DO: 使用之前能正确运行的assistant代码，比如 assistant_ticket_bot-3.py，然后改写为现在的股票查询assistant

@assistant_ticket_bot-3.py 基于stock_history_data.sql 改写成 股票查询助手，
这里MySQL的database=stock，编写新的 .py

```
...ery_assistant.py (new) +57 •
import pymysql
import pandas as pd

# 数据库连接配置
DB_CONFIG = {
    'host': 'localhost', # 数据库主机地址
    'user': 'root',      # 数据库用户名，请根据
    'password': 'your_password', # 数据库密码
    'database': 'stock', # 数据库名
    'charset': 'utf8mb4'
}

# 查询助手主类
class StockQueryAssistant:
```



```
import pymysql
import pandas as pd

# 数据库连接配置
DB_CONFIG = {
    'host': 'localhost', # 数据库主机地址
    'user': 'root',      # 数据库用户名，请根据实际情况修改
    'password': 'your_password', # 数据库密码，请根据实际情况修改
    'database': 'stock', # 数据库名
    'charset': 'utf8mb4'
}

# 查询助手主类
class StockQueryAssistant:
    def __init__(self, db_config):
        self.conn = pymysql.connect(**db_config)

    def query_by_stock_name(self, stock_name):
        """
        根据股票名称查询所有历史价格数据
        """
        sql = "SELECT * FROM stock_price WHERE stock_name = %s ORDER BY"
        df = pd.read_sql(sql, self.conn, params=[stock_name])
        return df
```

有些参数配置还要检查下，
替换为自己的MySQL信息

查询历史价格

@stock_query_assistant.py 需要实现的方式和 @assistant_ticket_bot-3.py 完全一样，即采用qwen-agent的方式，需要替换system_prompt里面的sql

你的需求是：将 `stock_query_assistant.py` 改造成和 `assistant_ticket_bot-3.py` 一样的 qwen-agent 方式，system_prompt 里描述 `stock_price` 表结构和字段，SQL执行工具要能自动可视化，并且支持终端和 WebUI 两种交互模式。

下面是主要改造点：

1. system_prompt 需描述 `stock_price` 表结构和字段（用你实际的表结构和注释）。
2. exc_sql 工具的 SQL 执行要连接你的阿里云 MySQL，数据库为 `stock`，表为 `stock_price`。
3. 终端和 WebUI 入口与 `assistant_ticket_bot-3.py` 保持一致。
4. 代码结构、可视化、函数注册等全部仿照 `assistant_ticket_bot-3.py`。

我将为你生成新的 `stock_query_assistant.py`，完全仿照上述风格。



```
# ===== 股票查询助手 system prompt 和函数描述 =====
system_prompt = """我是股票查询助手，以下是关于股票历史价格表 stock_price
-- 股票历史价格表
CREATE TABLE stock_price (
    id INT AUTO_INCREMENT PRIMARY KEY COMMENT '自增主键',
    stock_name VARCHAR(20) NOT NULL COMMENT '股票名称',
    ts_code VARCHAR(20) NOT NULL COMMENT '股票代码',
    trade_date VARCHAR(10) NOT NULL COMMENT '交易日期',
    open DECIMAL(15,2) COMMENT '开盘价',
    high DECIMAL(15,2) COMMENT '最高价',
    low DECIMAL(15,2) COMMENT '最低价',
    close DECIMAL(15,2) COMMENT '收盘价',
    vol DECIMAL(20,2) COMMENT '成交量',
    amount DECIMAL(20,2) COMMENT '成交额',
    UNIQUE KEY uniq_stock_date (ts_code, trade_date)
);
我将回答用户关于股票历史价格的相关问题。
每当 exc_sql 工具返回 markdown 表格和图片时，你必须原样输出工具返回的全部内
"""

functions_desc = [
    {
        "name": "exc_sql",
        "description": "对于生成的SQL，进行SQL查询",
        "parameters": {
            "type": "object",
```

检查实现的代码，
需要确认是否为
qwen-agent框架

查询历史价格

设置开场白问题：

- 查询2024年全年贵州茅台的收盘价走势
- 统计2024年4月国泰君安日均成交量
- 对比2024年中芯国际和贵州茅台的涨跌幅

查询2024年全年贵州茅台的收盘价走势



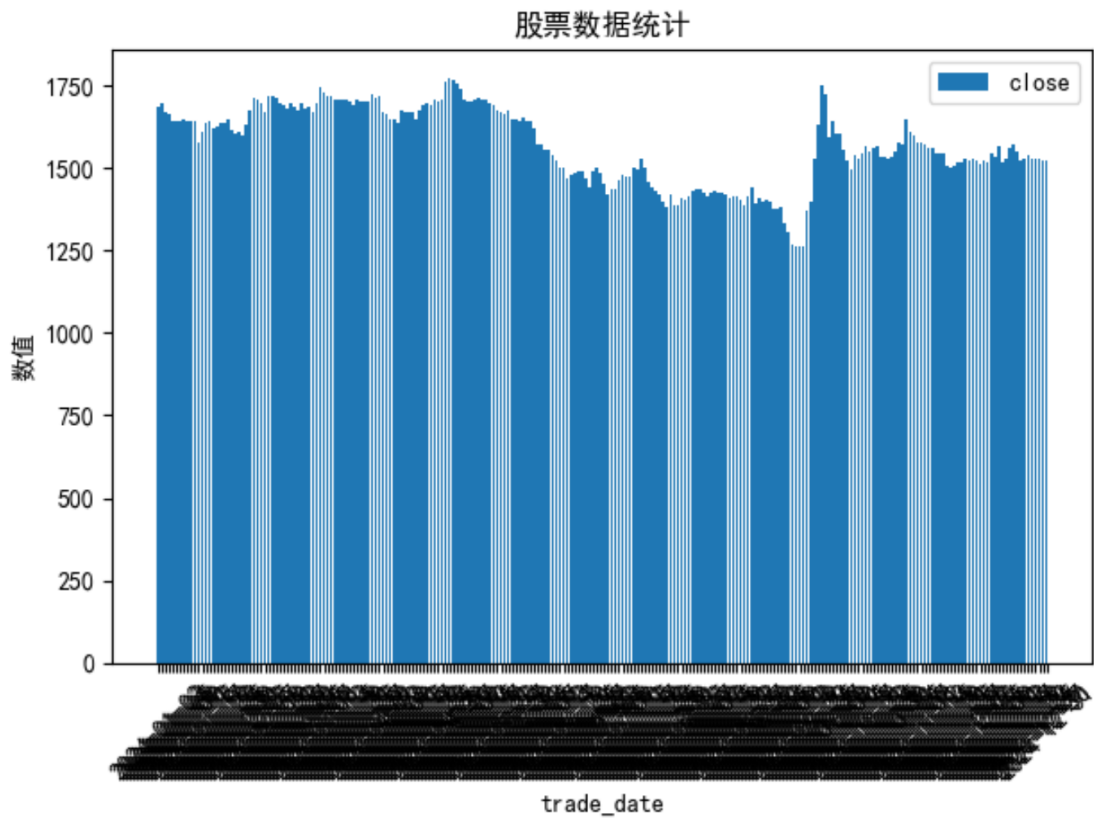
user

► Start calling tool "exc_sql" ...

► Finished tool calling.

以下是2024年全年贵州茅台的收盘价走势数据：

trade_date	close
2024-01-02	1685.01
2024-01-03	1694
2024-01-04	1669
2024-01-05	1663.36
2024-01-08	1643.99
2024-01-09	1641



Thinking：图表默认是用柱状图实现的，当数据量大的时候，没法完全显示，如何修改？

查询历史价格

在 ExcSQLTool 中的 call 中自动对提取到的数据 df 进行了可视化（柱状图）

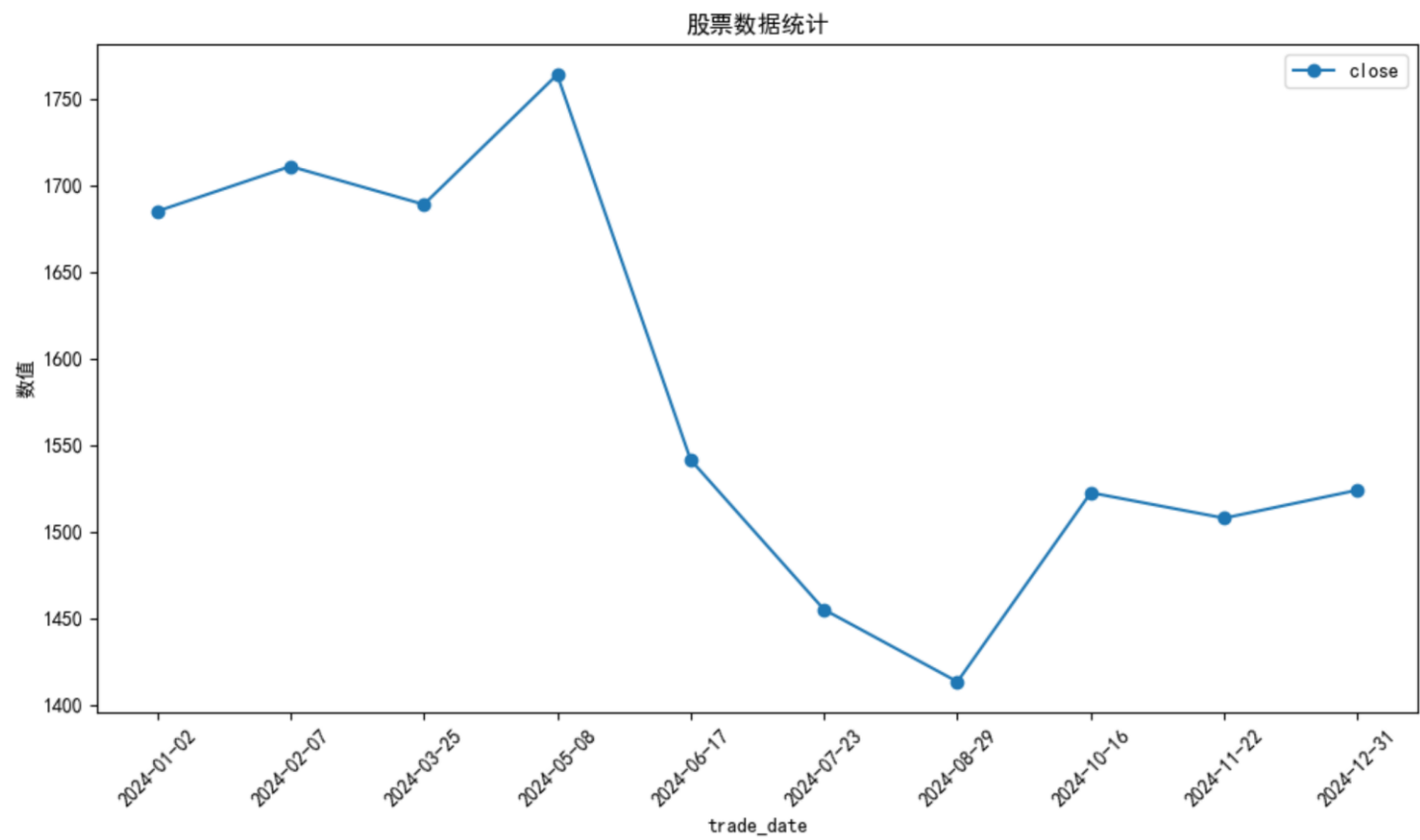
但如果 len(df) 较大，需要自动采用折线图进行呈现。且横坐标可以做一些筛选，比如选取 10 个点

改写的代码，写入到 stock_query_assistant-2.py

```
...assistant-2.py (new) +199 •
import os
import asyncio
from typing import Optional
import dashscope
from qwen_agent.agents import Assistant
from qwen_agent.gui import WebUI
import pandas as pd
from sqlalchemy import create_engine
from qwen_agent.tools.base import BaseTool,
import matplotlib.pyplot as plt
import io
import base64
import time
import numpy as np
```

```
# 如果数据点较多，自动采样10个点
if len(df_sql) > 20:
    idx = np.linspace(0, len(df_sql) - 1, 10, dtype=int)
    x = x.iloc[idx]
    df_plot = df_sql.iloc[idx]
    chart_type = 'line'
else:
    df_plot = df_sql
    chart_type = 'bar'
plt.figure(figsize=(10, 6))
for y_col in y_cols:
    if chart_type == 'bar':
        plt.bar(df_plot[x_col], df_plot[y_col], label=str(y_col))
    else:
        plt.plot(df_plot[x_col], df_plot[y_col], marker='o', label=
plt.xlabel(x_col)
plt.ylabel('数值')
plt.title('股票数据统计')
plt.legend()
```

查询历史价格



Thinking: 如果assistant只是将历史数据展示, 可以让AI对数据进行总结, 回复给用户, 方便用户了解这个查询结果的整体情况, 如何实现?

查询历史价格

- 1、完善 system_prompt, 添加：当我查询到数据的时候，可以对这个数据进行简单的总结。
- 2、在exc_sql工具回复的时候，添加df的 describe描述

在exc_sql工具回复的时候，添加df的 describe描述，即df.describe(), 这样方便AI进行总结回复给用户

```
...query_assistant-2.py +3 -1 •
try:
    df = pd.read_sql(sql_input, engine)
    md = df.head(10).to_markdown(index=False)
    desc_md = df.describe().to_markdown()
    # 自动创建目录
    save_dir = os.path.join(os.path.dirname(__file__), 'data')
    os.makedirs(save_dir, exist_ok=True)
    generate_smart_chart_png(df, save_dir)
    img_path = os.path.join('image_show', f'{int(time.time()*1000)}.png')
    img_md = f'![图表]({img_path})'
    return f"{md}\n\n{img_md}"
    return f"{md}\n\n{desc_md}\n\n{img_md}"
except Exception as e:
```

```
desc_md = df.describe().to_markdown()
# 自动创建目录
save_dir = os.path.join(os.path.dirname(__file__), 'data')
os.makedirs(save_dir, exist_ok=True)
filename = f'stock_{int(time.time()*1000)}.png'
save_path = os.path.join(save_dir, filename)
# 智能选择可视化方式
generate_smart_chart_png(df, save_path)
img_path = os.path.join('image_show', filename)
img_md = f'![图表]({img_path})'
return f"{md}\n\n{img_md}"
return f"{md}\n\n{desc_md}\n\n{img_md}"
except Exception as e:
    return f"SQL执行或可视化出错: {str(e)}"
```


查询历史价格

查询2024年全年贵州茅台的收盘价走势

► Start calling tool "exc_sql" ...

► Finished tool calling.

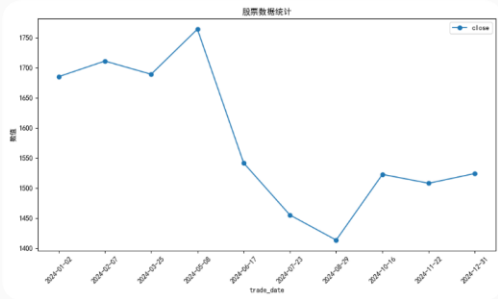
查询结果显示，2024年全年贵州茅台的收盘价走势如下：

收盘价数据（部分示例）：

交易日期	收盘价
2024-01-02	1685.01
2024-01-03	1694
2024-01-04	1669
2024-01-05	1663.36
2024-01-08	1643.99
2024-01-09	1641
2024-01-10	1641.5
2024-01-11	1646.57
2024-01-12	1643.06
2024-01-15	1640

统计信息：

指标	值
数量	242
平均值	1569.6
标准差	116.148
最小值	1261
第25百分位	1490.54
中位数	1568.25
第75百分位	1673.74
最大值	1770



从以上数据可以看出，贵州茅台在2024年的股价整体呈现波动趋势。如果您需要更详细的分析或图表，请告诉我！

可根据自己的情况，进行system_prompt的调整，指导AI回复哪些内容

查询历史价格

TO DO：测试更多的测试用例，查看assistant回复的情况

统计2024年4月国泰君安日均成交量



► Start calling tool "exc_sql" ...

► Finished tool calling.

统计结果显示，2024年4月国泰君安日均成交量为 345,812。

统计信息：

指标	值
数量	1
平均值	345812
标准差	nan
最小值	345812
第25百分位	345812
中位数	345812
第75百分位	345812
最大值	345812

无可视化数据

从以上数据可以看出，2024年4月国泰君安日均成交量为345,812。如果需要进一步分析，请随时告诉我！

通过测试发现：当结果只有一行的时候，不需要进行统计信息，以及可视化图表

查询历史价格

当结果只有一行的时候，不需要进行统计信息，也不需要可视化图表

```
stock_query_assistant-2.py +3 ✓
try:
    df = pd.read_sql(sql_input, engine)
    md = df.head(10).to_markdown(index=False)
    if len(df) == 1:
        return md
    desc_md = df.describe().to_markdown(index=False)
    # 自动创建目录
    save_dir = os.path.join(os.path.dirname(__file__), 'data')
    os.makedirs(save_dir, exist_ok=True)
    df.to_csv(os.path.join(save_dir, 'data.csv'), index=False)

if __name__ == '__main__':
    app_gui() # 默认启动Web界面
```

已完成优化：现在当 SQL 查询结果只有一行时，exc_sql 工具只返回表格，不再返回统计信息和可视化图片。

如需进一步细化逻辑或自定义返回内容，欢迎随时告知！



统计2024年4月国泰君安日均成交量



► Start calling tool "exc_sql" ...

► Finished tool calling.

根据查询结果，2024年4月国泰君安的日均成交量为 345,812。

对比分析

对比2024年中芯国际和贵州茅台的涨跌幅

对比2024年中芯国际和贵州茅台的涨跌幅

► Start calling tool "exc_sql" ...

► Finished tool calling.

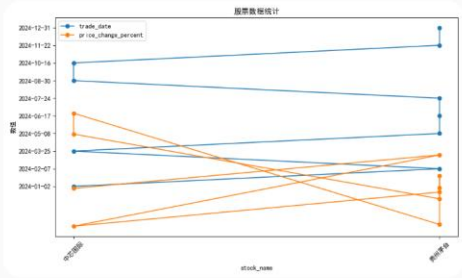
以下是2024年中芯国际和贵州茅台的涨跌幅数据：

涨跌幅数据表

stock_name	trade_date	price_change_percent
中芯国际	2024-01-02	-0.113165
贵州茅台	2024-01-02	-1.74869
中芯国际	2024-01-03	-0.700095
贵州茅台	2024-01-03	0.766755
贵州茅台	2024-01-04	-1.4176
中芯国际	2024-01-04	-1.18095
贵州茅台	2024-01-05	0.122191
中芯国际	2024-01-05	-2.26087
中芯国际	2024-01-08	-2.15458
贵州茅台	2024-01-08	-1.02408

统计信息

	price_change_percent
count	484
mean	0.125944
std	2.51767
min	-11.8449
25%	-0.909716
50%	-0.0169655
75%	0.839471
max	18.9182



从数据可以看出，中芯国际和贵州茅台在2024年的涨跌幅情况。如果需要进一步分析或特定日期的数据，请告诉我！

Thinking：从这里你能看出什么问题？如何进行思考和改进？

对比分析

Thinking: 从这里你能看出什么问题? 如何进行思考和改进?

- 1) 数据只展示了前10行 => 导致AI分析是对1月2日和1月8日进行的涨跌幅对比分析
- 2) 统计信息和图表这里可以忽略

在 ExcSQLTool 中的 call 中的 md 会提取 df 的前 10 行, 这里可以改成提取 df 的前 5 行 + df 的后 5 行, 这样可以让 AI 看到更完整的信息

另外可以 args 中传入一个参数为是否需要可视化, 这样让 AI 大模型可以自主判断, 传入这个参数

当不需要进行可视化的时候, 后面的 describe() 和 可视化都不需要进行



```
try:
    df = pd.read_sql(sql_input, engine)
    md = df.head(10).to_markdown(index=False)
    # 前5行+后5行拼接展示
    if len(df) > 10:
        md = pd.concat([df.head(5), df.tail(5)]).to_markdown(index=False)
    else:
        md = df.to_markdown(index=False)
    # 只返回表格
    if len(df) == 1:
        return md
    need_visualize = args.get('need_visualize', True)
    if not need_visualize:
        return md
```

对比分析

完成后需要检查下代码逻辑，比如这里虽然设置了need_visualize的参数判断，但是在工具的parameters中并没有该参数

在 ExcSQLTool的parameters增加 need_visualize, 默认是显示为True。这里说明下如果是对比分析，设置为False，不需要让大模型进行可视化

```
parameters = [{
    'name': 'sql_input',
    'type': 'string',
    'description': '生成的SQL语句',
    'required': True
},
{
    'name': 'need_visualize',
    'type': 'boolean',
    'description': '是否需要可视化和统计信息', 默认True。如果是对比分析
    'required': False,
    'default': True
}]
```

对比分析

对比2024年中芯国际和贵州茅台的涨跌幅



user

▼ Start calling tool "exc_sql" ...

```
{
  "sql_input": "SELECT stock_name, trade_date, (close - open) / open * 100 AS price_change_percent
FROM stock_price WHERE trade_date BETWEEN '2024-01-01' AND '2024-12-31' AND stock_name IN ('中芯国际', '贵州茅台');",
  "need_visualize": false
}
```

▼ Finished tool calling.

stock_name	trade_date	price_change_percent
中芯国际	2024-01-02	-0.113165
贵州茅台	2024-01-02	-1.74869
中芯国际	2024-01-03	-0.700095
贵州茅台	2024-01-03	0.766755
贵州茅台	2024-01-04	-1.4176
中芯国际	2024-12-27	0.754288
中芯国际	2024-12-30	2.78468
贵州茅台	2024-12-30	-0.584757
中芯国际	2024-12-31	-4.42424
贵州茅台	2024-12-31	-0.091779

根据2024年中芯国际和贵州茅台的每日涨跌幅数据，我们可以看到它们在不同日期的表现。为了更清晰地对比这两只股票的整体表现，以下是对它们在2024年的涨跌幅进行统计分析：

涨跌幅统计

- 中芯国际
 - 最高涨幅：2.78468%（2024-12-30）
 - 最低跌幅：-4.42424%（2024-12-31）
 - 平均涨跌幅：需要进一步计算。
- 贵州茅台
 - 最高涨幅：0.766755%（2024-01-03）
 - 最低跌幅：-1.74869%（2024-01-02）
 - 平均涨跌幅：需要进一步计算。

如果您需要更详细的统计信息或可视化图表，请告诉我！

df已经改成了显示前5行和后5行，但是2024年涨跌幅计算并不正确，如何完善？

对比分析

利用qwen-agent的知识库能力，可以作为上下文补充，即给大模型提供Tips或相关的示例说明

编写 faq.txt 如下：

Q1：对比2000年股票A和股票B的涨跌幅

A1：

我需要先找到股票A在2000年第一天和最后一天的价格，再找到股票B在2000年第一天和最后一天的价格，

然后计算股票A的涨跌幅 = (最后一天价格 - 第一天价格) / 第一天价格 * 100%

同样计算股票B的涨跌幅 = (最后一天价格 - 第一天价格) / 第一天价格 * 100%

然后对比这两只股票的涨跌幅。

```
try:
    bot = Assistant(
        llm=llm_cfg,
        name='股票查询助手',
        description='股票历史价格查询与分析',
        system_message=system_prompt,
        function_list=['exc_sql'],
        files = ['./faq.txt']
    )
    print("助手初始化成功！")
    return bot
```

然后在 Assistant注册时候，添加上 files = ['./faq.txt']

对比分析

对比2024年中芯国际和贵州茅台的涨跌幅



user

► Start calling tool "exc_sql" ...

► Finished tool calling.

根据提供的数据，以下是2024年中芯国际和贵州茅台的涨跌幅分析：

中芯国际

- 2024年1月2日：收盘价为 52.96
- 2024年12月31日：收盘价为 49.5

涨跌幅计算：

$$\text{涨跌幅} = \frac{49.5 - 52.96}{52.96} \times 100\% \approx -6.53\%$$

贵州茅台

- 2024年1月1日：收盘价为 1530
- 2024年12月31日：收盘价为 1524

涨跌幅计算：

$$\text{涨跌幅} = \frac{1524 - 1530}{1530} \times 100\% \approx -0.39\%$$

对比结果

- 中芯国际 在2024年的涨跌幅为 -6.53%
- 贵州茅台 在2024年的涨跌幅为 -0.39%

从结果来看，中芯国际在2024年的表现不如贵州茅台，跌幅更大。



股票查询助

计算逻辑对了，但是结果不正确，原因是什么？

可以观察下这里展示的df => 如何完善？

stock_name	trade_date	close
中芯国际	2024-01-02	52.96
中芯国际	2024-01-03	52.48
中芯国际	2024-01-04	51.88
中芯国际	2024-01-05	50.58
中芯国际	2024-01-08	49.5
贵州茅台	2024-12-25	1530
贵州茅台	2024-12-26	1527.79
贵州茅台	2024-12-27	1528.97
贵州茅台	2024-12-30	1525
贵州茅台	2024-12-31	1524

对比分析

完善 faq.txt 如下：

Q1：对比2000年股票A和股票B的涨跌幅

A1：

我需要先找到股票A在2000年第一天和最后一天的价格，再找到股票B在2000年第一天和最后一天的价格，

然后计算股票A的涨跌幅 = (最后一天价格 - 第一天价格) / 第一天价格 * 100%

同样计算股票B的涨跌幅 = (最后一天价格 - 第一天价格) / 第一天价格 * 100%

然后对比这两只股票的涨跌幅。

查询股票撰写SQL的时候，需要按照 trade_date 升序排序，这样可以看到不同股票在同一天价格对比。

对比分析

对比2024年中芯国际和贵州茅台的涨跌幅



user

► Start calling tool "exc_sql" ...

► Finished tool calling.

根据提供的数据，以下是2024年中芯国际和贵州茅台的涨跌幅分析：

中芯国际

- 2024年1月2日：收盘价为 52.96
- 2024年12月31日：收盘价为 49.5

涨跌幅计算：

$$\text{涨跌幅} = \frac{49.5 - 52.96}{52.96} \times 100\% \approx -6.53\%$$

贵州茅台

- 2024年1月1日：收盘价为 1530
- 2024年12月31日：收盘价为 1524

涨跌幅计算：

$$\text{涨跌幅} = \frac{1524 - 1530}{1530} \times 100\% \approx -0.39\%$$

对比结果

- 中芯国际 在2024年的涨跌幅为 -6.53%
- 贵州茅台 在2024年的涨跌幅为 -0.39%

从结果来看，中芯国际在2024年的表现不如贵州茅台，跌幅更大。



股票查询助

计算逻辑对了，但是结果不正确，原因是什么？

可以观察下这里展示的df => 如何完善？

stock_name	trade_date	close
中芯国际	2024-01-02	52.96
中芯国际	2024-01-03	52.48
中芯国际	2024-01-04	51.88
中芯国际	2024-01-05	50.58
中芯国际	2024-01-08	49.5
贵州茅台	2024-12-25	1530
贵州茅台	2024-12-26	1527.79
贵州茅台	2024-12-27	1528.97
贵州茅台	2024-12-30	1525
贵州茅台	2024-12-31	1524

对比分析

对比2024年中芯国际和贵州茅台的涨跌幅



► Start calling tool "exc_sql" ...

► Finished tool calling.

根据提供的数据，以下是2024年中芯国际和贵州茅台的涨跌幅计算：

中芯国际

- 2024年1月2日收盘价：52.96
- 2024年12月31日收盘价：94.62
- 涨跌幅： $(94.62 - 52.96) / 52.96 * 100\% \approx 78.67\%$

贵州茅台

- 2024年1月2日收盘价：1685.01
- 2024年12月31日收盘价：1524
- 涨跌幅： $(1524 - 1685.01) / 1685.01 * 100\% \approx -9.56\%$

对比分析

- 中芯国际在2024年表现出显著的上涨趋势，涨幅约为78.67%。
- 贵州茅台则略有下跌，跌幅约为9.56%。

从整体来看，中芯国际在2024年的表现优于贵州茅台。

热点新闻

Thinking: 如何给Assistant配置热点新闻能力?

可以使用 bing-cn-mcp-server 或 tavily

在modelscope MCP广场中找到 Bing中文搜索服务 或 tavily

这里推荐使用: tavily

通过SSE URL连接服务

i Hosted MCP 服务的 SSE URL是为您分配的专属连接地址, 为敏感信息, 请勿对外泄漏!

有效期

长期有效

连接

通过SSE URL连接服务

i Hosted MCP 服务的 SSE URL是为您分配的专属连接地址, 为敏感信息, 请勿对外泄漏!

```
{  "mcpServers": {    "tavily-mcp": {      "type": "sse",      "url": "https://mcp.api-inference.modelscope.net/..."    }  }}
```

试用

配置

使用魔搭免费云端资源部署

我们发现modelscope提供的tavily-mcp 通过SSE方式返回结果不对
因此需要使用npx方式调用

热点新闻

获取tavily api key <https://app.tavily.com/home>

API Keys +

The key is used to authenticate your requests to the [Research API](#). To learn more, see the [documentation](#) page.

NAME	TYPE	USAGE	KEY	OPTIONS
default	dev	157	tvly-dev-*****	   

```
# MCP 工具配置
tools = [{
  "mcpServers": {
    # "tavily-mcp": {
    #   "type": "sse",
    #   "url": "https://mcp.api-inference.modelscope.net"
    # }
    "tavily-mcp": {
      "command": "npx",
      "args": ["-y", "tavily-mcp@0.1.4"],
      "env": {
        "TAVILY_API_KEY": "tvly-dev-*****"
      },
      "disabled": false,
      "autoApprove": []
    }
    # "bing-cn-mcp-server": {
    #   "type": "sse",
    #   "url": "https://mcp.api-inference.modelscope.cn/"
    # }
  }
}, 'exc_sql']
```

贵州茅台 最近新闻




user

- ▶ Start calling tool "tavily-mcp-tavily-search" ...
- ▼ Finished tool calling.
- Detailed Results:
- Title: 贵州茅台(*Sh600519*) 值得关注的是最近 茅台 开瓶率明显上升，零售端购买热情增长，黄牛因流动性集中抛售，但经... - 雪球

URL: <https://xueqiu.com/1055241569/339166138>

Content: 贵州茅台(*sh600519*) 值得关注的是最近 茅台 开瓶率明显上升，零售端购买热情增长，黄牛因流动性集中抛售，但经销商稳定。其他白酒企业产品市场关注度和声音变小，茅台关注度上升。 ... "是近期新闻媒体市场调查的报道，大家可以去搜!
- Title: 大跌后关注茅台触底信号，食品饮料ETF天弘（159736）早盘逆势上涨_腾讯新闻

URL: <https://news.qq.com/rain/a/20250618A03JFZ00>

Content: 成分股中， 贵州茅台 更是从5月份高点1645元一度跌至1401.18元，期间累计跌幅约15%。消息面上，白酒板块处于承压阶段，飞天茅台价格持续走低
- Title: 飞天茅台线上售价跌破1850元大关 - 澎湃新闻

URL: https://m.thepaper.cn/newsDetail_forward_30992664

Content: 面对今年来价格下行压力，贵州茅台已悄然采取行动。肖竹青向澎湃新闻透露，其于4月中下旬了解到，目前贵州茅台全国42家直营店停止销售500ml飞天茅台，仅提供公斤装、100ml小瓶装、精品茅台及生肖酒。
- Title: 飞天茅台电商促销价跌破1850元：经销商称线下影响有限 高端白酒如何破局？_贵州茅台(600519)股吧_东方财富网股吧

打卡：股票查询助手



打造股票查询助手，包括：

- 股票历史价格的查询（折线图、柱状图呈现）
- 多支股票的对比分析
- 热点新闻

查询2024年全年贵州茅台的收盘价走势



► Start calling tool "exc_sql" ...

► Finished tool calling.

2024年全年贵州茅台的收盘价走势如下：

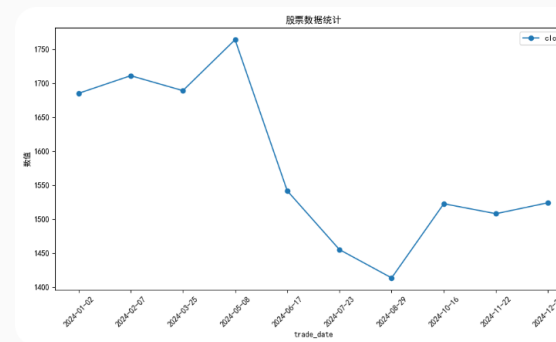
- 2024年1月2日：1685.01
- 2024年12月31日：1524

从数据来看，贵州茅台在2024年的整体趋势是略有下跌。尽管期间有波动，但最终收盘价低于年初水平。

走势分析：

- 最高点：1770（具体日期未提供）
- 最低点：1261（具体日期未提供）
- 平均收盘价：1569.6
- 标准差：116.15（表明价格波动较大）

图表说明：



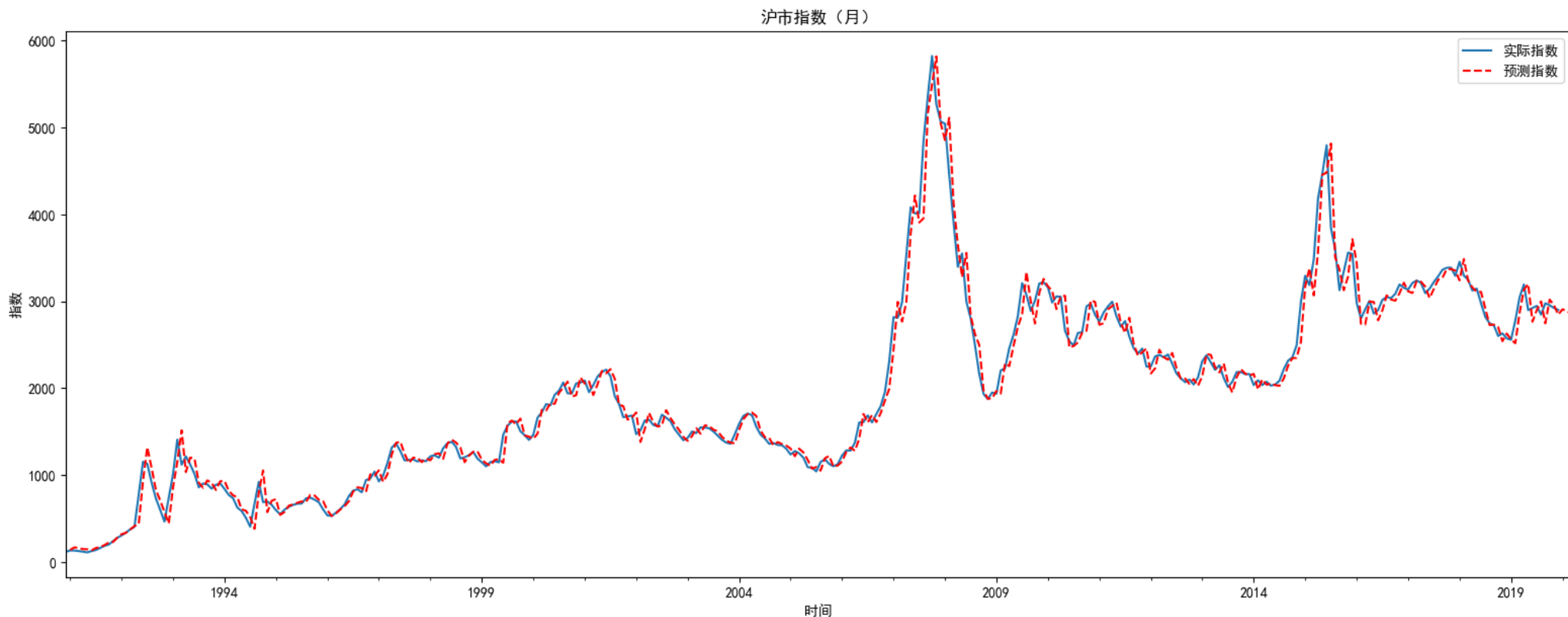
该图表展示了贵州茅台在2024年的收盘价走势，可以更直观地观察其价格变化趋势和波动情况。

趋势预测

ARIMA（自回归积分滑动平均模型）是经典的时间序列预测方法：结合趋势（AR）、差分（I）和噪声（MA）建模，适合非平稳序列（如股价），通过差分消除非平稳性。

ARIMA的局限：**假设线性关系，难以捕捉股价的非线性特征（如突发事件、市场情绪）**

注意事项：股价受外部因素（政策、新闻）影响大，纯ARIMA预测需谨慎，需结合基本面和市场分析。



趋势预测

我想使用ARIMA，对未来N天的股票价格进行预测，编写工具 arima_stock

传入股票的ts_code，以及预测的天数 n，这里ts_code是必填

预测的时候需要先从远端阿里云MySQL中获取该股票的数据，获取截止到今天的前一年的历史价格，然后使用ARIMA进行建模 (5,1,5)，并对未来n天的价格进行预测

在@stock_query_assistant-2.py 的基础上进行编写，写入到 stock_query_assistant-3.py

```
stock_query_assistant-3.py +313 ✓  
  
import os  
import asyncio  
from typing import Optional  
import dashscope  
from qwen_agent.agents import Assistant  
from qwen_agent.gui import WebUI  
import pandas as pd  
from sqlalchemy import create_engine  
from qwen_agent.tools.base import BaseTool, register_tool  
import matplotlib.pyplot as plt  
import io  
import base64  
import time  
  
import numpy as np
```

ARIMA 股票预测工具已为你设计好，主要功能如下：

- 工具名：arima_stock
- 参数：ts_code（必填，股票代码），n（必填，预测天数）
- 实现逻辑：
 1. 自动从阿里云MySQL获取该股票近一年历史收盘价（截止到今天前一天）。
 2. 用 ARIMA(5,1,5) 进行建模。
 3. 预测未来 n 天的收盘价，返回预测表格和预测折线图。
 4. 若历史数据不足30天，自动提示无法建模。

你可以直接在 stock_query_assistant-3.py 中使用该工具，支持 WebUI 和终端模式。

如需进一步自定义ARIMA参数、支持多股票批量预测、或增加更多可视化细节，欢迎随时告知！

趋势预测

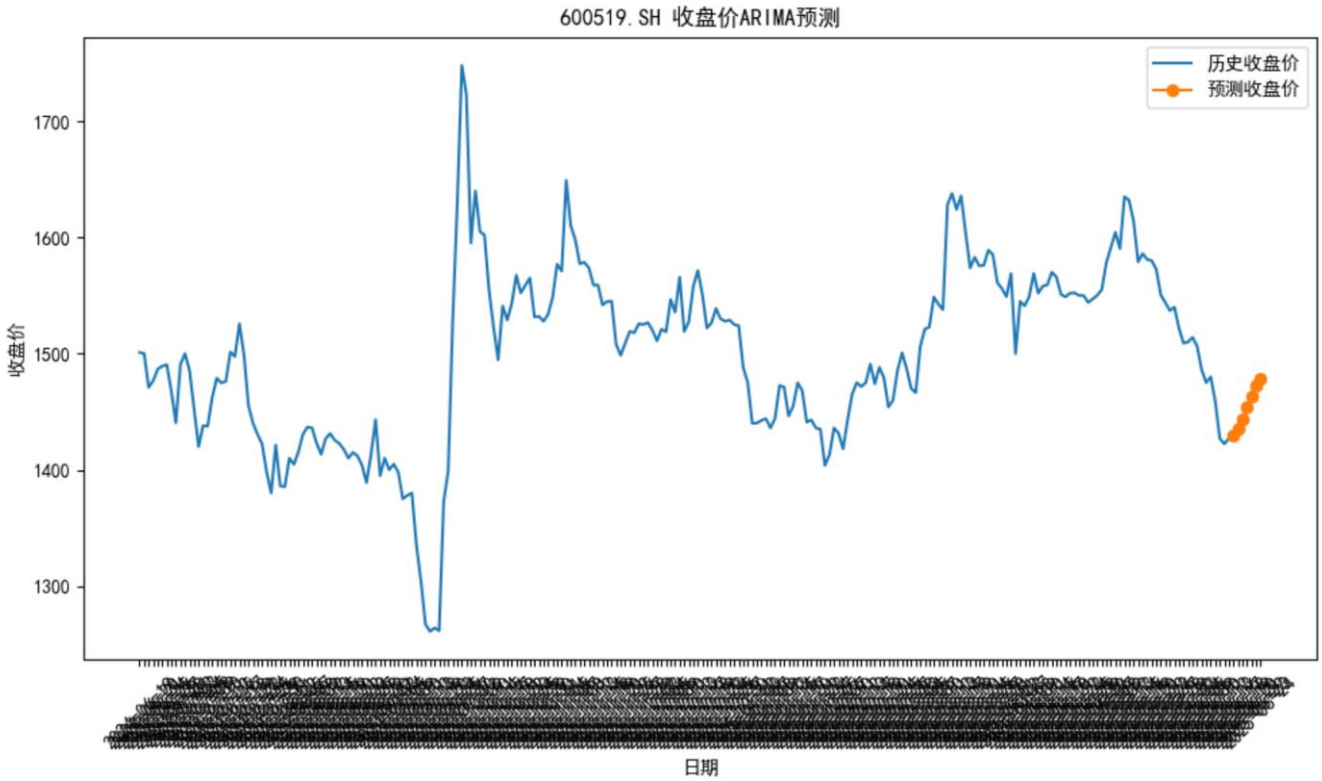
预测贵州茅台未来7天的收盘价

▼ Start calling tool "arima_stock" ...

```
{"ts_code": "600519.SH", "n": 7}
```

► Finished tool calling.

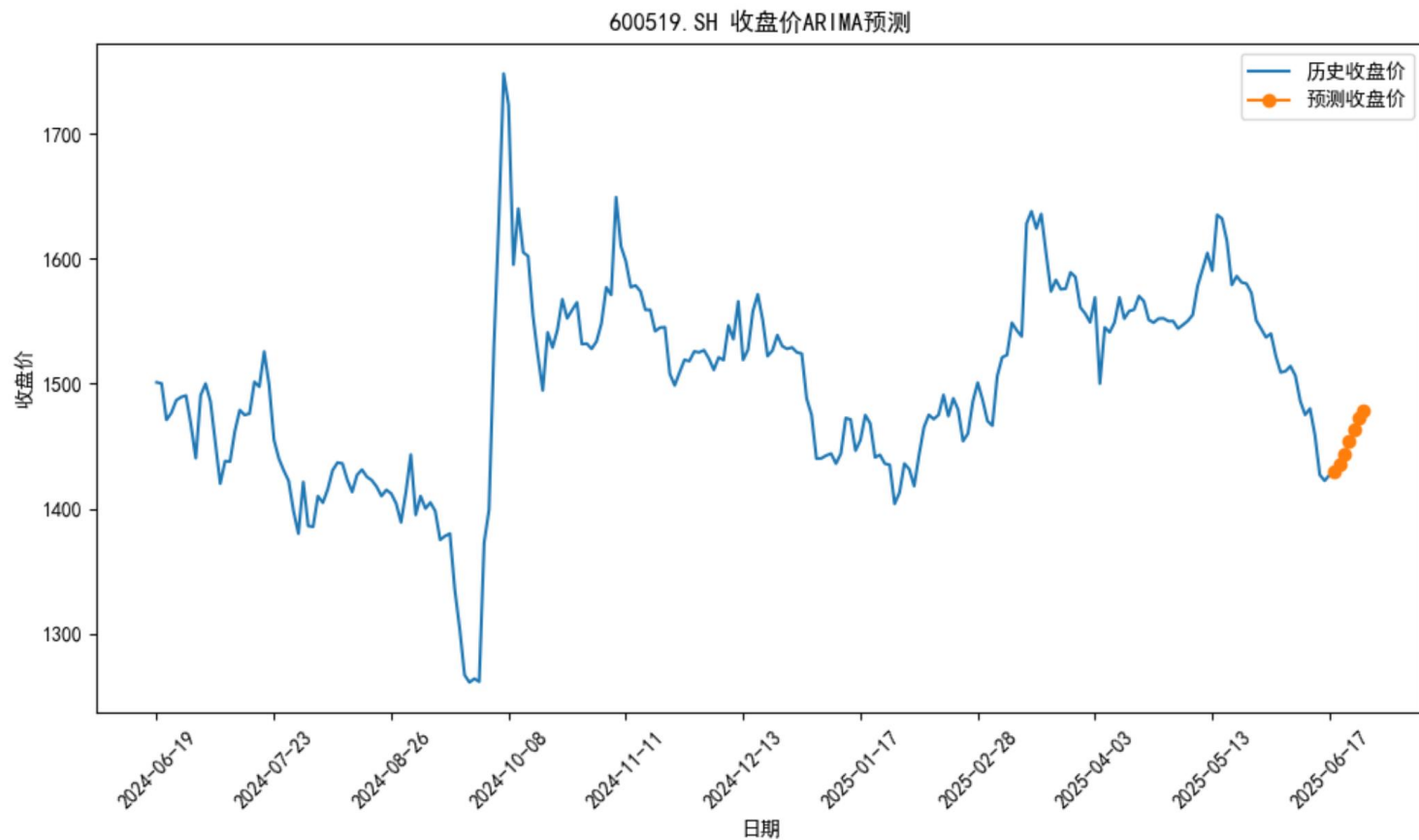
预测日期	预测收盘价
2025-06-18	1429.52
2025-06-19	1435.13
2025-06-20	1443.18
2025-06-21	1453.56
2025-06-22	1463.65
2025-06-23	1472.75
2025-06-24	1478.92



Thinking：有什么问题，如何进行调整？

趋势预测

arima_stock得到了预测图，但是横坐标太密了，可以根据df的长度进行判断，选择其中的一些点进行显示即可。



趋势预测

在 `faq.txt` 中添加趋势预测回复示例：

Q2: 预测xxx股票未来x天的价格

A2:

我需要使用`arima_stock`进行预测，然后回复用户，这里要对股票的未来价格进行解释，比如后续价格是怎样的。

- 1、对files内容进行更新，需要删除 workspace，然后再运行，会更新 workspace的缓存
- 2、可以在 `C:\Users\cheny\AppData\Roaming\Python\Python311\site-packages\qwen_agent` 中对 Assistant类添加代码，了解 rag prompt的拼接过程，以便更精确的控制
- 3、如果files 影响力弱，可以在 `system_prompt`中进行添加

趋势预测

预测贵州茅台未来7天的收盘价

► Start calling tool "arima_stock" ...

► Finished tool calling.

根据ARIMA模型的预测，贵州茅台（股票代码：600519.SH）未来7天的收盘价预测如下：

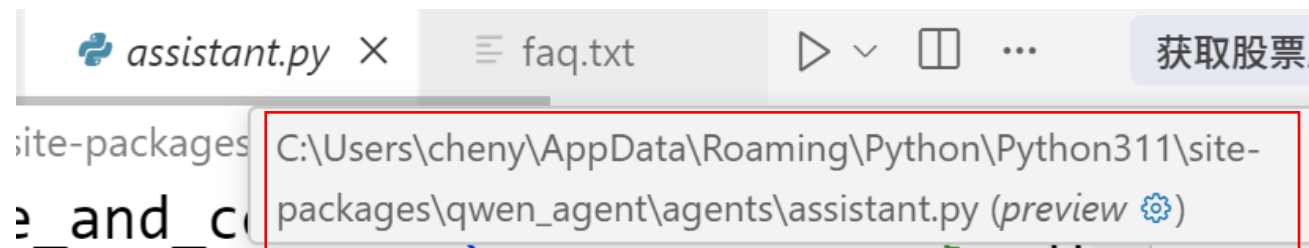
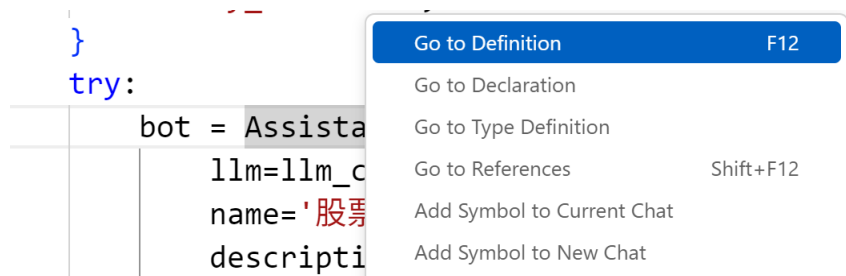
预测日期	预测收盘价
2025-06-18	1429.52
2025-06-19	1435.13
2025-06-20	1443.18
2025-06-21	1453.56
2025-06-22	1463.65
2025-06-23	1472.75
2025-06-24	1478.92

从预测结果来看，贵州茅台的收盘价在未来7天内预计将呈现逐步上升的趋势。价格将持续走高，但涨幅逐渐减缓。



趋势预测

Thinking: 如何了解到你的Assistant类的源代码在哪里?



趋势预测

TO DO: 找到本地的 assistant.py, 可以添加过程打印代码, 了解rag prompt的拼接过程

```
100 def _run(self,
101         messages: List[Message],
102         lang: Literal['en', 'zh'] = 'en',
103         knowledge: str = '',
104         **kwargs) -> Iterator[List[Message]]:
105     """Q&A with RAG and tool use abilities.
106
107     Args:
108         knowledge: If an external knowledge string is provided,
109         it will be used directly without retrieving information from files in messages.
110
111     """
112
113     new_messages = self._prepend_knowledge_prompt(messages=messages, lang=lang, knowledge=knowledge)
114     # ===== 以下为调试打印, 方便用户查看拼接后的完整prompt =====
115     print("\n===== 拼接后的完整prompt [system prompt部分] =====")
116     for msg in new_messages:
117         if msg.role == SYSTEM:
118             print(msg.content)
119             print("===== END system prompt =====\n")
120             break
121     # ===== 结束打印 =====
122     return super()._run(messages=new_messages, lang=lang, **kwargs)
```


趋势预测

再次运行bot的时候，可以看到 rag prompt拼接的结果

```
===== 拼接后的完整prompt (system prompt部分) =====
我是股票查询助手，以下是关于股票历史价格表 stock_price 的字段，我可能会编写对应的SQL，对数据进行查询
-- 股票历史价格表
CREATE TABLE stock_price (
    id INT AUTO_INCREMENT PRIMARY KEY COMMENT '自增主键',
    stock_name VARCHAR(20) NOT NULL COMMENT '股票名称',
    ts_code VARCHAR(20) NOT NULL COMMENT '股票代码',
    trade_date VARCHAR(10) NOT NULL COMMENT '交易日期',
    open DECIMAL(15,2) COMMENT '开盘价',
    high DECIMAL(15,2) COMMENT '最高价',
    low DECIMAL(15,2) COMMENT '最低价',
    close DECIMAL(15,2) COMMENT '收盘价',
    vol DECIMAL(20,2) COMMENT '成交量',
    amount DECIMAL(20,2) COMMENT '成交额',
    UNIQUE KEY uniq_stock_date (ts_code, trade_date)
);
我将回答用户关于股票历史价格的相关问题。
每当 exc_sql 工具返回 markdown 表格和图片时，你必须原样输出工具返回的全部内容（包括图片 markdown），不要只总结表格，也不要省略图片。这样用户才能直接看到表格和图片。
如果是预测未来价格，需要对未来的价格进行详细的解释说明，比如价格将持续走高，或价格将相对平稳， 或价格将持续走低。
```

```
# 知识库

## 来自 [文件](faq.txt) 的内容:

...

Q1: 对比2000年股票A和股票B的涨跌幅
A1:
我需要先找到股票A在2000年第一天和最后一天的价格，再找到股票B在2000年第一天和最后一天的价格，
然后计算股票A的涨跌幅 = (最后一天价格 - 第一天价格) / 第一天价格 * 100%
同样计算股票B的涨跌幅 = (最后一天价格 - 第一天价格) / 第一天价格 * 100%
然后对比这两只股票的涨跌幅。

查询股票撰写SQL的时候，需要按照 trade_date 升序排序，这样可以看到不同股票在同一天价格对比。trade_date的格式类似：2024-01-01
==
Q2: 预测xxx股票未来x天的价格
A2:
我需要使用arima_stock进行预测，然后回复用户，这里要对股票的未來价格进行解释，比如后续价格是怎样的。

...
```

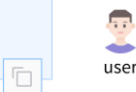
打卡：股票趋势预测



开发股票趋势预测，在股票查询助手的基础上增加趋势预测：

- 时间序列方法，比如ARIMA对未来一段时间的某支股票价格进行预测

预测贵州茅台未来7天的收盘价



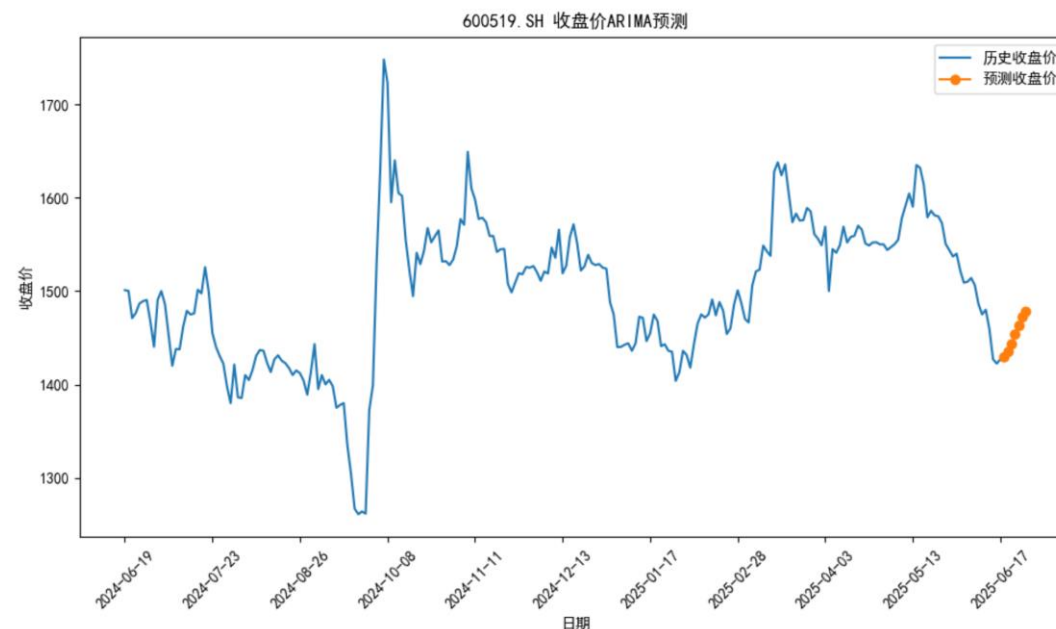
► Start calling tool "arima_stock" ...

► Finished tool calling.

根据ARIMA模型的预测，贵州茅台（股票代码：600519.SH）未来7天的收盘价预测如下：

预测日期	预测收盘价
2025-06-18	1429.52
2025-06-19	1435.13
2025-06-20	1443.18
2025-06-21	1453.56
2025-06-22	1463.65
2025-06-23	1472.75
2025-06-24	1478.92

从预测结果来看，贵州茅台的收盘价在未来7天内预计将呈现逐步上升的趋势。价格将持续走高，但涨幅逐渐减缓。



股票价格异常检测

股票价格异常检测

布林带（Bollinger Bands）：

由约翰·布林格在1980年代开发的一种技术分析工具，用于衡量价格波动性、识别超买/超卖状态。

它基于统计学中的标准差（ σ ）和移动平均线（MA），广泛应用于股票、外汇、加密货币等金融市场。



股票价格异常检测

Thinking: 布林带的构成是怎样的?

布林带由三条线组成:

中轨 (Middle Band) : 通常为20日简单移动平均线 (SMA) , 代表中期趋势。

$$\text{中轨} = \text{SMA}_{20}(\text{收盘价})$$

上轨 (Upper Band) : 中轨 + 2倍标准差 (2σ) , 反映价格波动的上限。

$$\text{上轨} = \text{SMA}_{20} + 2 \times \text{标准差}_{20}$$

下轨 (Lower Band) : 中轨 - 2倍标准差 (2σ) , 反映价格波动的下限。

$$\text{下轨} = \text{SMA}_{20} - 2 \times \text{标准差}_{20}$$

说明: 默认使用20日周期+ 2σ , 但可根据交易风格调整 (如短线交易可能用10日+ 1.5σ) 。

股票价格异常检测

Thinking: 布林带的作用是什么?

(1) 衡量波动性

带宽 (Bandwidth) 收窄: 标准差变小 → 波动性降低, 可能预示盘整或即将突破 (如“布林带收缩”后的大行情)。

带宽扩大: 标准差变大 → 波动性增强, 趋势可能加速。

(2) 识别超买/超卖

价格触及上轨: 可能**超买** (但强势趋势中可能持续贴近上轨)。

价格触及下轨: 可能**超卖** (但下跌趋势中可能持续贴近下轨)。

(3) 趋势判断

价格沿上轨运行 → **强势上涨趋势**。

价格沿下轨运行 → **强势下跌趋势**。

价格穿越中轨 → 可能趋势反转 (需结合成交量等确认)。

生产中的SPC

生产中的SPC分析

SPC分析（Statistical Process Control）

- 统计过程控制，对生产过程进行分析，及时发现系统性因素出现的潜在问题，达到控制质量的目的
- 使用条件：只要问题用数字表示，就可以采用SPC分析
- 实施阶段：

1) 分析阶段

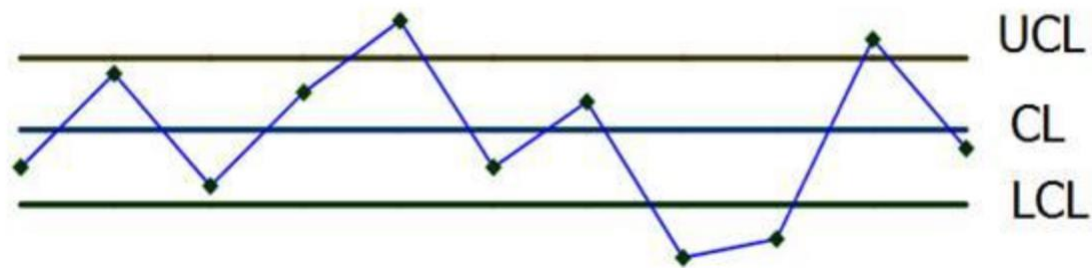
确保生产是在各要素无异常的情况下进行

用生产过程收集的数据计算控制界限，做成分析用控制图，检验生产过程是否处于统计稳态，过程能力是否足够

2) 监控阶段

使用控制图进行监控

生产过程的数据及时绘制到控制图上，控制图中点的波动情况可以显示出过程受控或失控，如果发现失控，需要找到原因，尽快消除影响



UCL: Upper Control Limit

CL: Control Line

LCL: Lower Control Limit

生产中的SPC分析

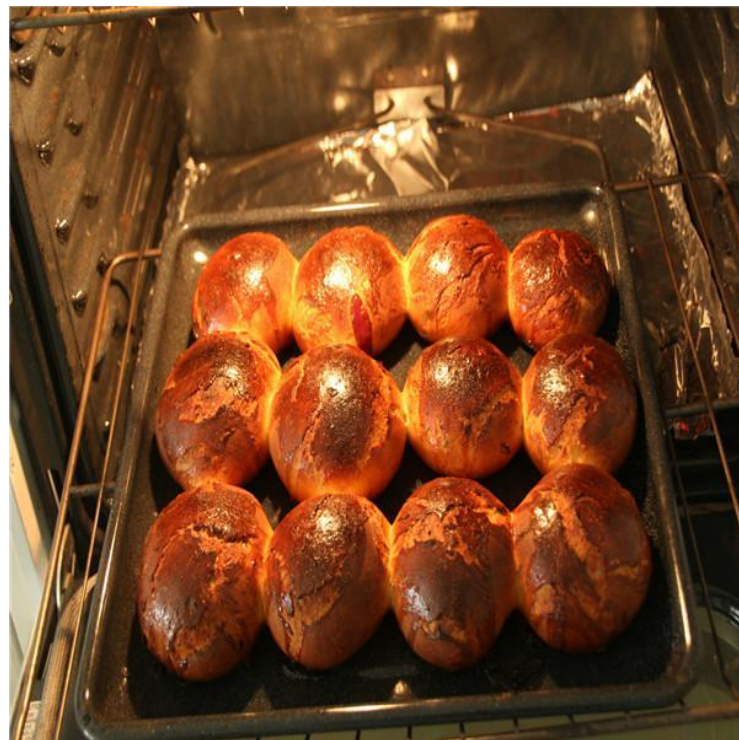
Thinking: 假设你开了一家面包店，对于生产的质量管控如何使用SPC分析？

1) 量产前

在面包店试营业期，对面包进行抽样试吃，用于确定烤箱、配方、烘烤温度、时间等是否做出美味的面包

2) 量产后

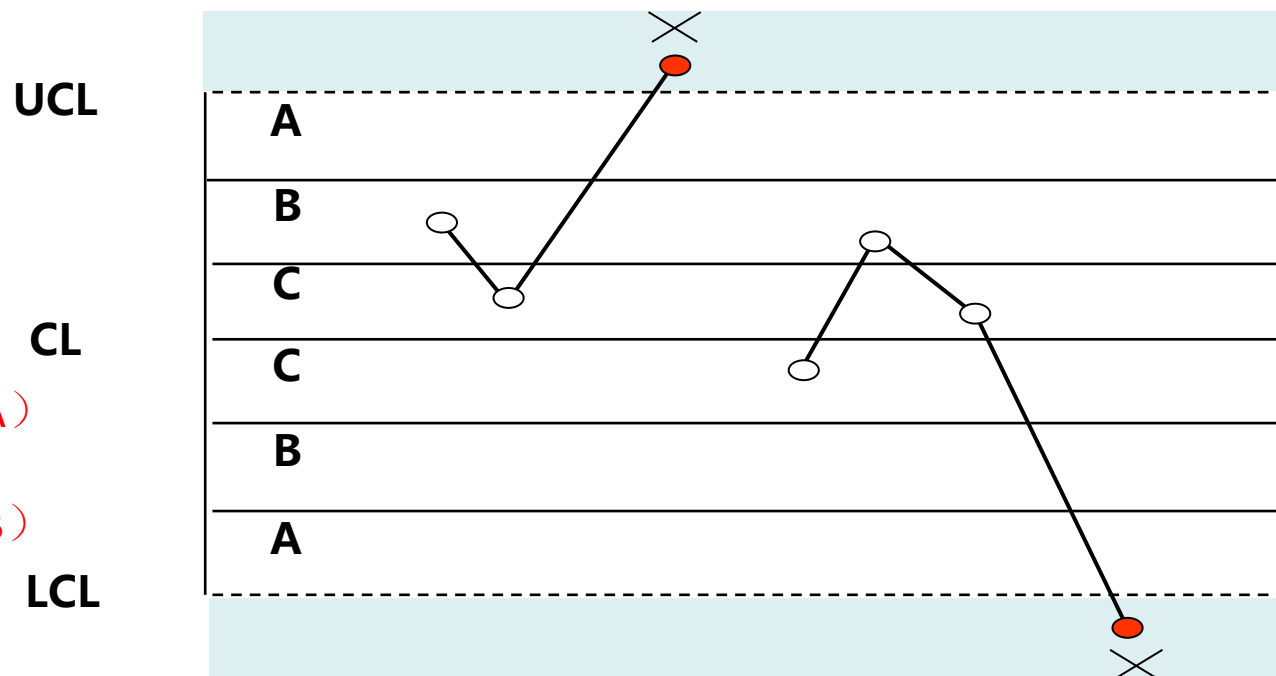
对每天做的面包进行抽样试吃，用于确定设备的过程控制是否发生异常，一旦发现控制图的异常（基于1个或多个准则），可以采取纠正措施进行纠正，避免后续的生产发生质量问题



SPC控制图判异准则

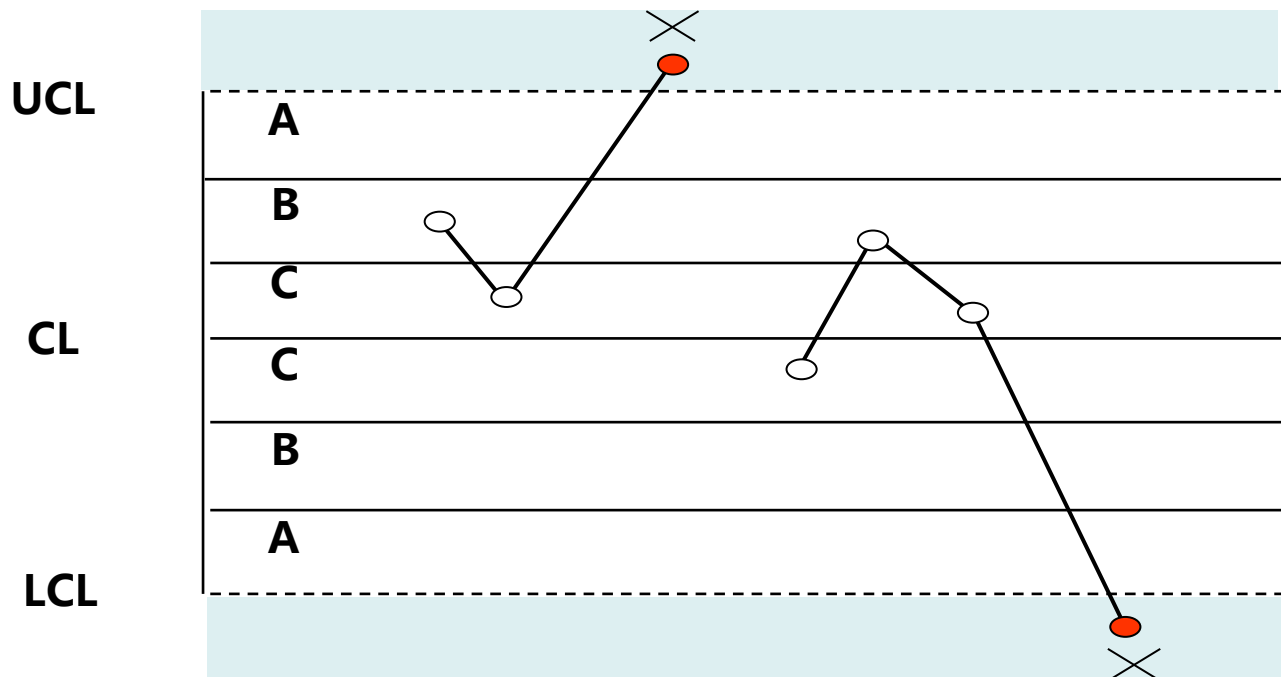
控制图判断异常的准则：

- 1) 1个点子落在A区以外（1点界外）
- 2) 连续9点落在中心线同一侧（9点单边）
- 3) 连续6点递增或递减（6连串）
- 4) 连续14点中相邻点子总是上下交替（14升降）
- 5) 连续3点中有2点落在中心线同一侧B区以外（2/3A）
- 6) 连续5点中有4点落在中心线同一侧C区以外（4/5B）
- 7) 连续15点落在中心线同两侧C区之内（15C）
- 8) 连续8点落在中心线两侧且无1点在C区中（8缺C）

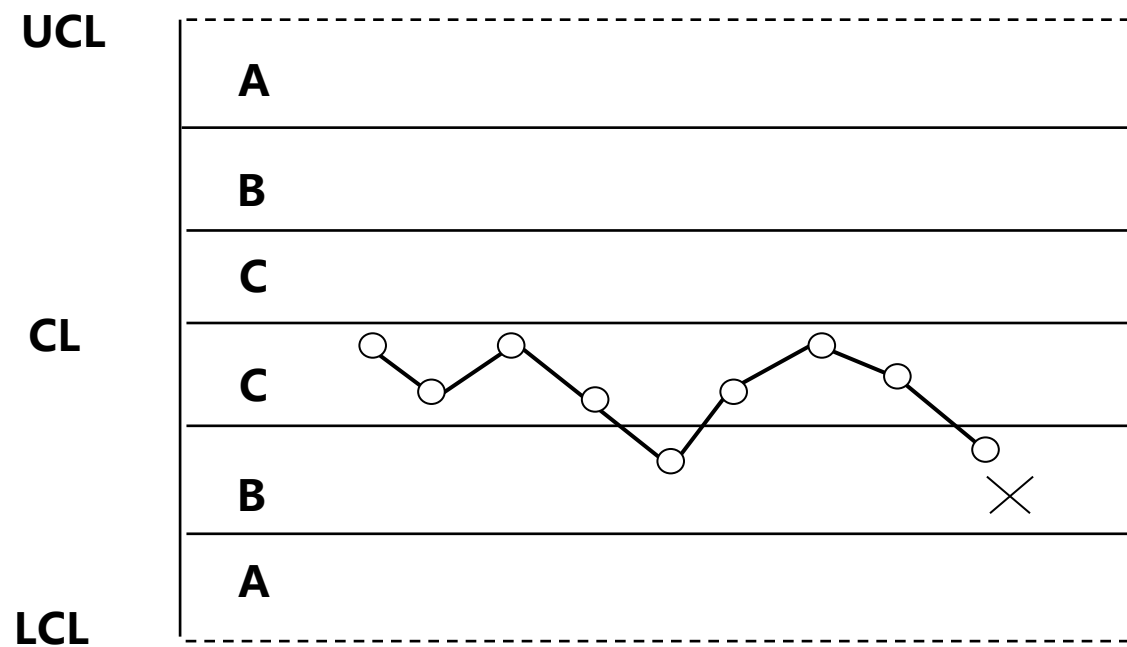


SPC控制图判异准则

1) 1个点子落在A区以外 (1点界外)

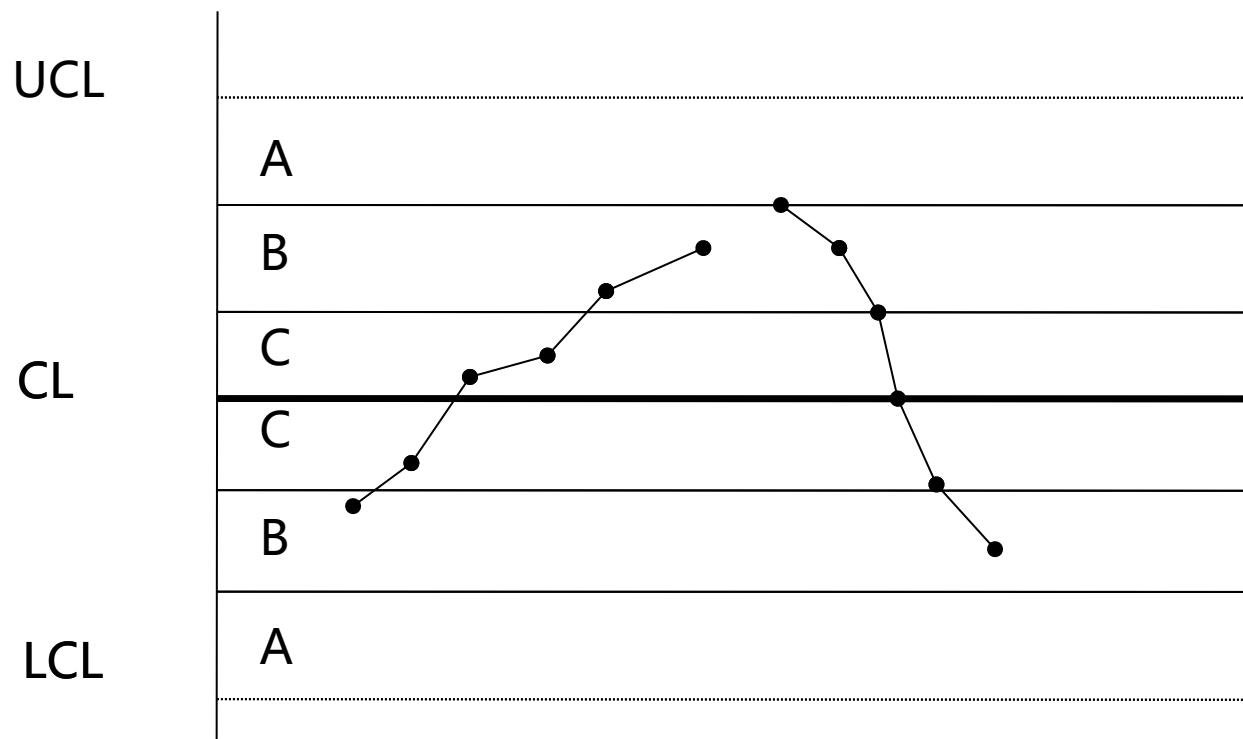


2) 连续9点落在中心线同一侧 (9点单边)

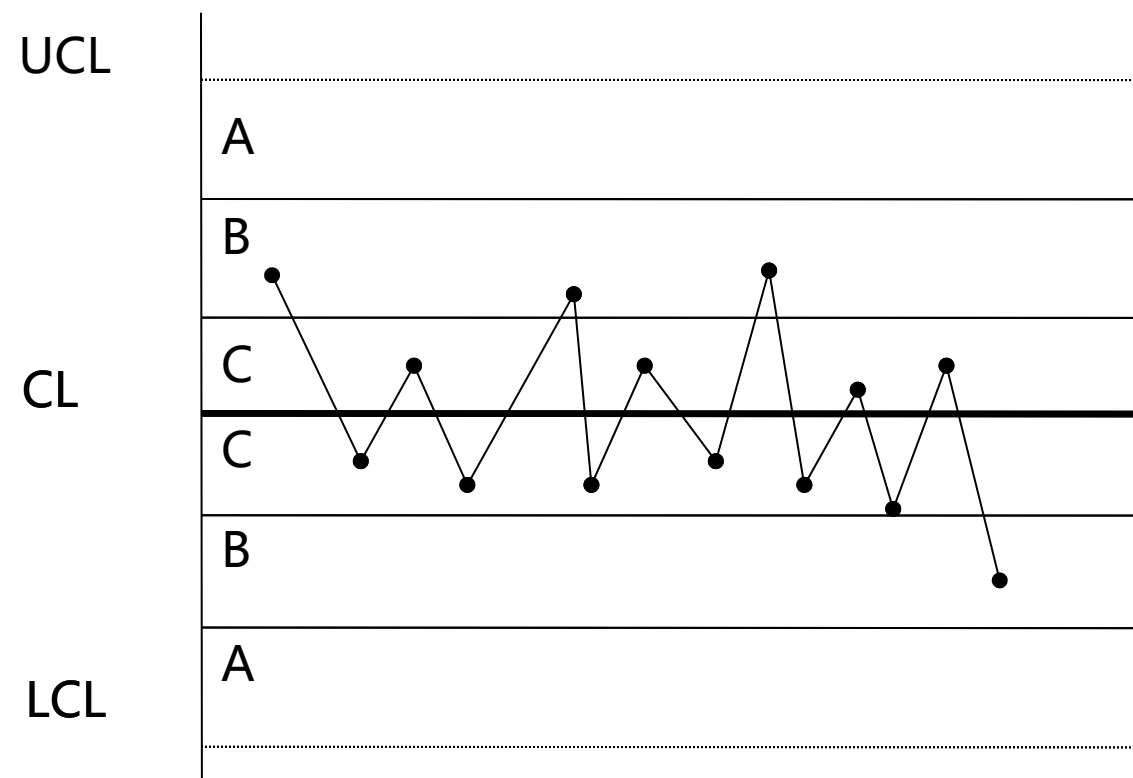


SPC控制图判异准则

3) 连续6点递增或递减 (6连串)

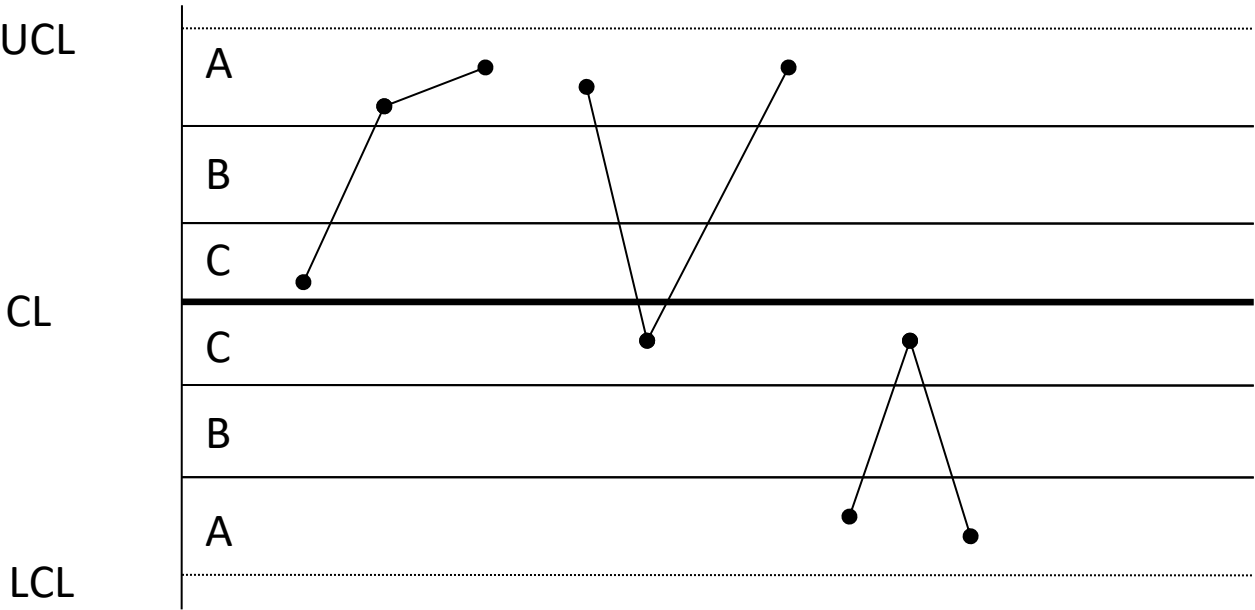


4) 连续14点中相邻点子总是上下交替 (14升降)

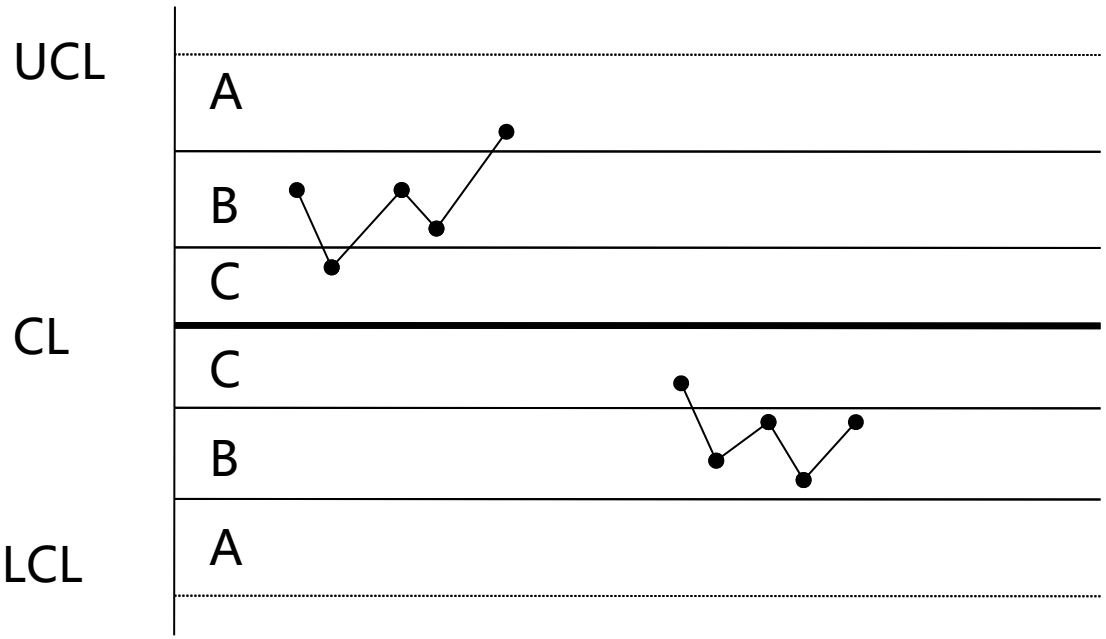


SPC控制图判异准则

5) 连续3点中有2点落在中心线同一侧B区以外 (2/3A)

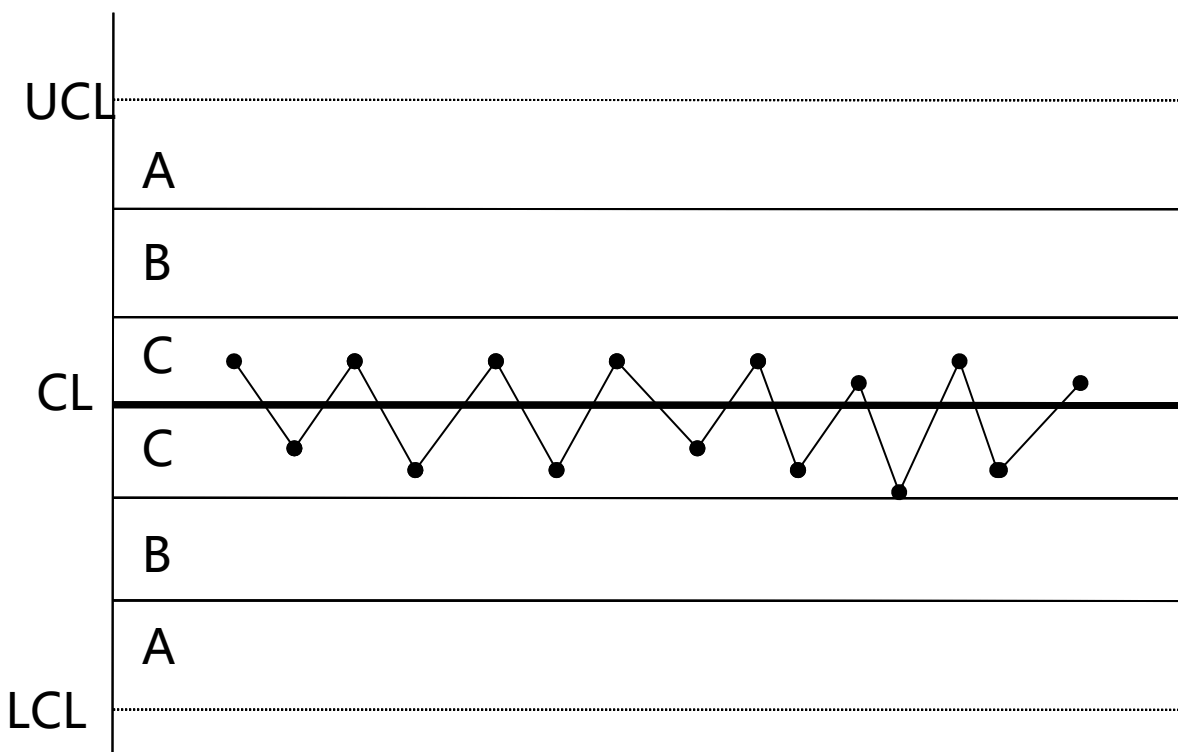


6) 连续5点中有4点落在中心线同一侧C区以外 (4/5B)

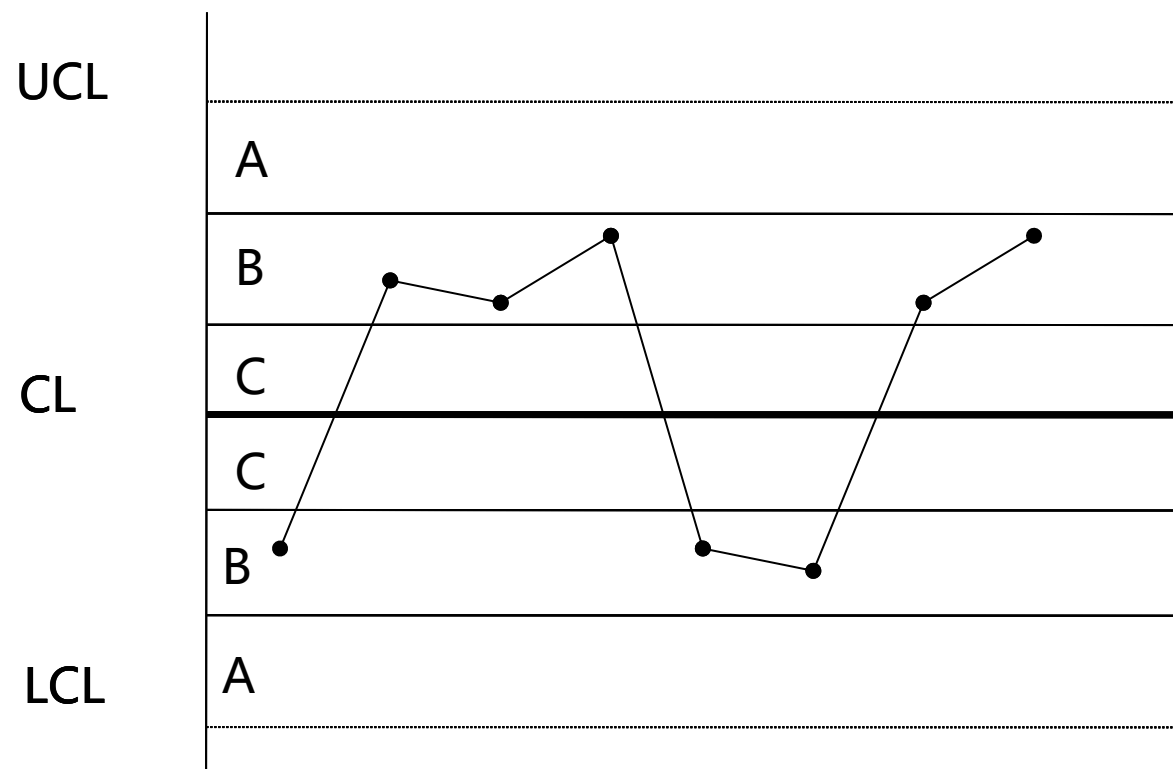


SPC控制图判异准则

7) 连续15点落在中心线同两侧C区之内 (15C)



8) 连续8点落在中心线两侧且无1点在C区中 (8缺C)



SPC原则：在质量控制中，通常使用 3σ 作为异常阈值（涵盖99.7%的正态分布数据），而 2σ （约95%覆盖）用于预警。

股票价格异常检测

我想对某支股票的异常点进行检测，使用boll布林带，用使用20日周期+2 σ 进行检测，默认检测过去1年的超买和超卖日期。

也可以让用户自定义范围，检测这段时间的超买和超卖点

编写 boll_detection工具。这里应该也要先从远端阿里云MySQL中获取该股票的数据，然后再进行检测。

在@stock_query_assistant-3.py 的基础上进行编写，写入到 stock_query_assistant-4.py

stock_query_assistant-4.py +269 ✓

```
import os
import asyncio
from typing import Optional
import dashscope
from qwen_agent.agents import Assistant
from qwen_agent.gui import WebUI
import pandas as pd
from sqlalchemy import create_engine
from qwen_agent.tools.base import BaseTool, register_tool
import matplotlib.pyplot as plt
import io
import base64
import time
import numpy as np
```

```
{
  "name": "boll_detection",
  "description": "对指定股票(ts_code)的收盘价进行布林带异常点检测，默认检测过去1年，也",
  "parameters": {
    "type": "object",
    "properties": {
      "ts_code": {
        "type": "string",
        "description": "股票代码，必填"
      },
      "start_date": {
        "type": "string",
        "description": "检测起始日期，格式YYYY-MM-DD，选填"
      },
      "end_date": {
        "type": "string",
        "description": "检测结束日期，格式YYYY-MM-DD，选填"
      }
    }
  },
  "required": ["ts_code"]
}
```

股票价格异常检测

检测贵州茅台近一年超买超卖点

▼ Start calling tool "boll_detection" ...

```
{ "ts_code": "600519.SH", "start_date": "2023-04-07", "end_date": "2024-04-07" }
```

► Finished tool calling.

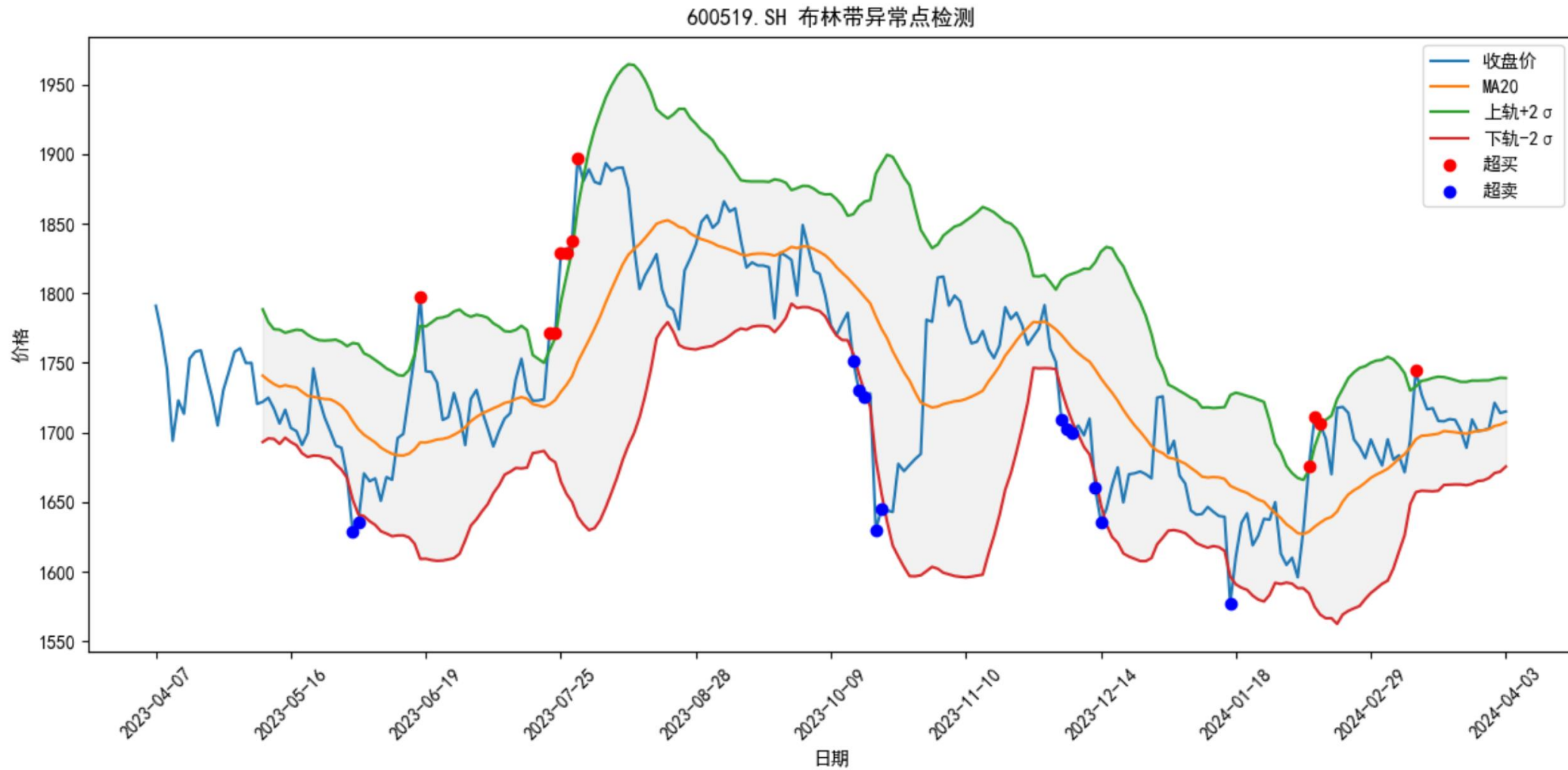
超买日期

trade_date	close
2023-06-16	1797.69
2023-07-21	1771.3
2023-07-24	1771.15
2023-07-25	1828.55
2023-07-26	1828.55
2023-07-27	1838.03
2023-07-28	1897
2024-02-06	1676
2024-02-07	1710.99
2024-02-08	1706
2024-03-12	1745

超卖日期

trade_date	close
2023-05-31	1628.9
2023-06-01	1635.92
2023-10-13	1751
2023-10-16	1729.9
2023-10-17	1726
2023-10-19	1630
2023-10-20	1645
2023-12-05	1709
2023-12-06	1703
2023-12-07	1700
2023-12-13	1660
2023-12-14	1635.9
2024-01-17	1577

股票价格异常检测

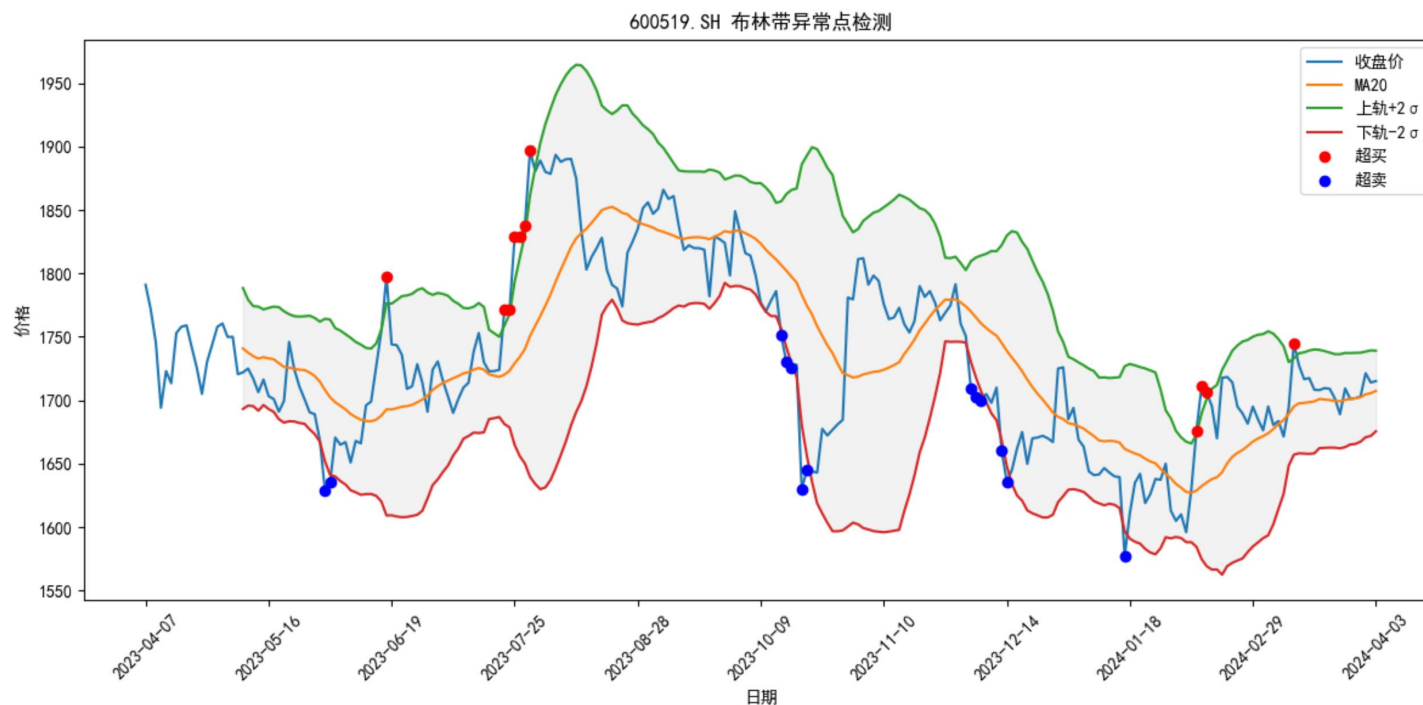


打卡：股票异常点检测

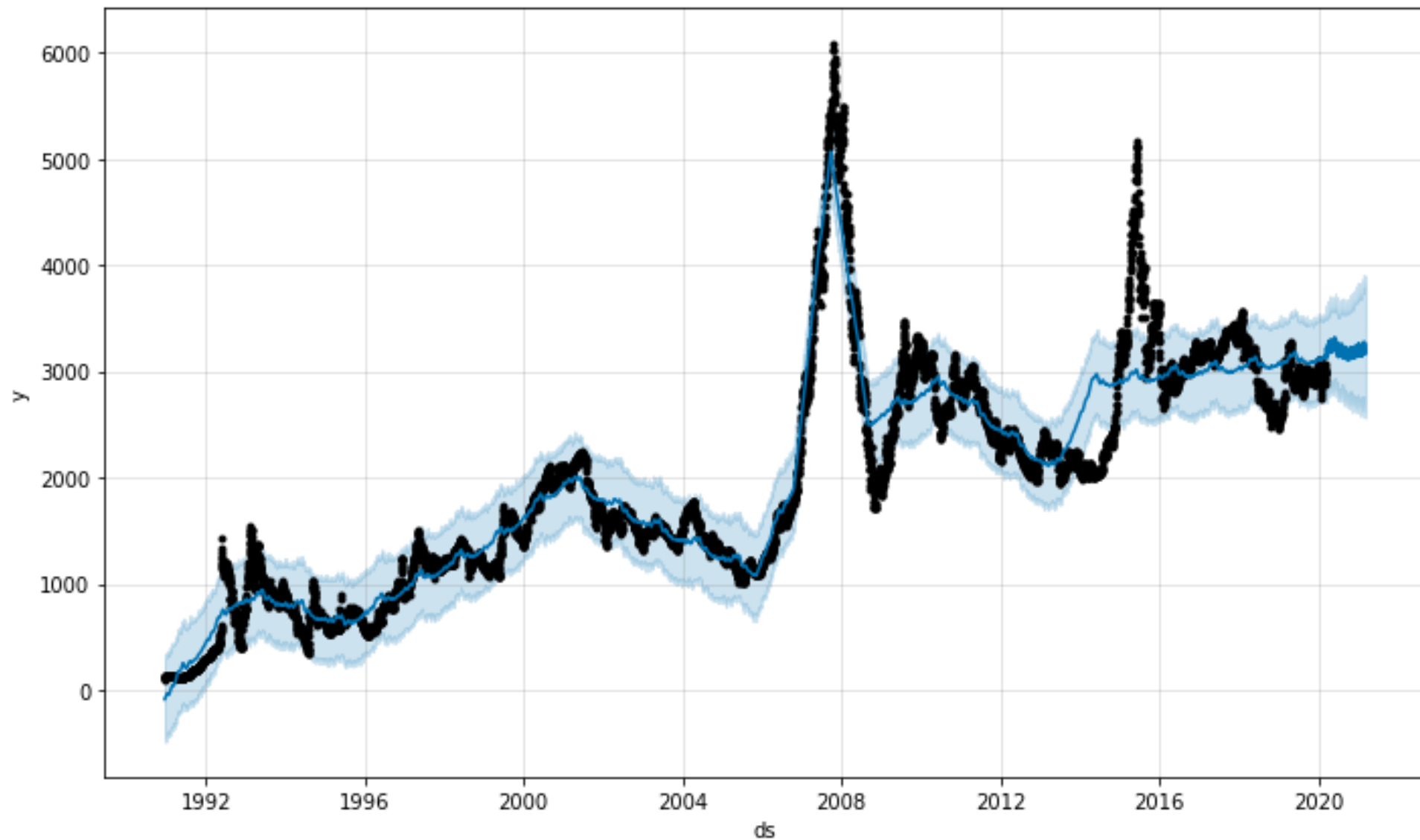


开发股票异常点检测，比如识别超买和超卖点：

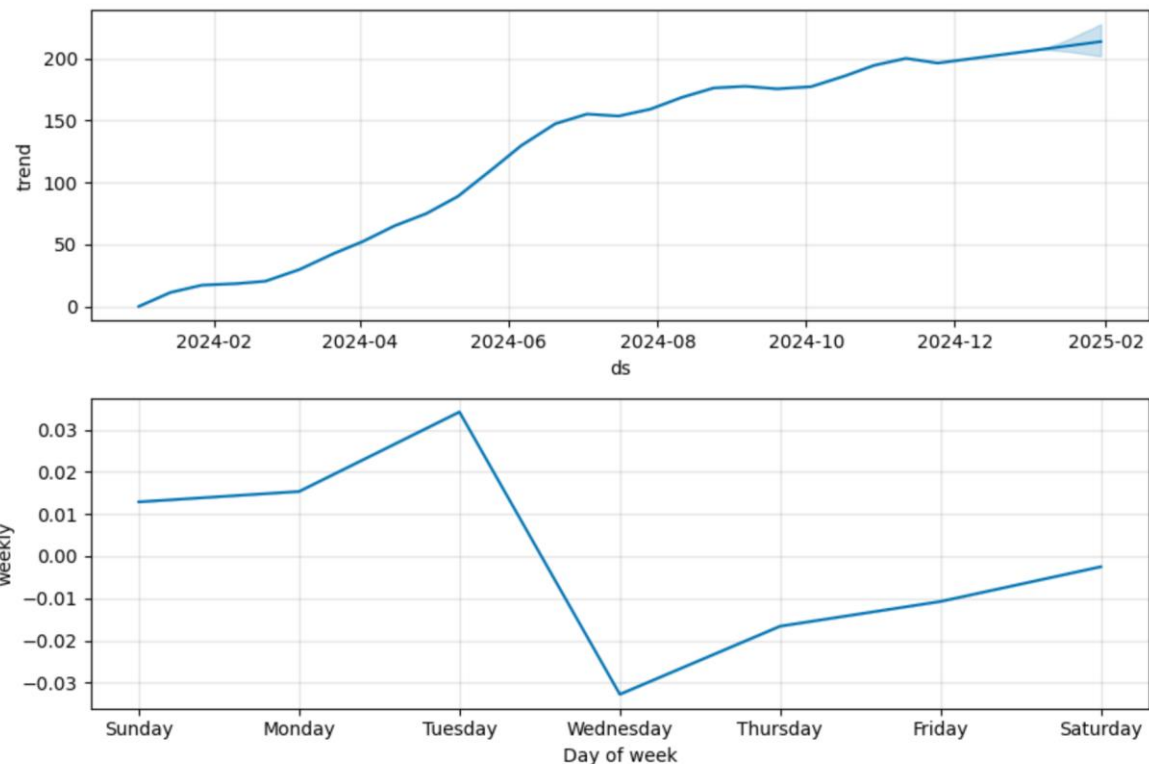
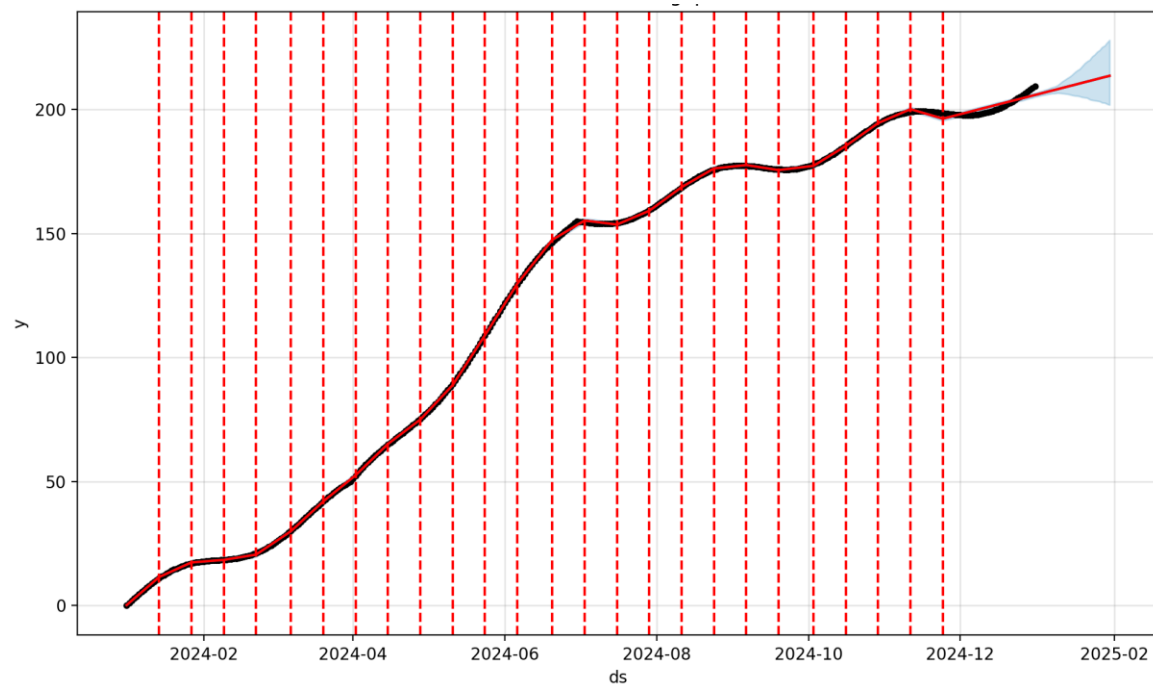
- 可以使用boll带检测方法，设置MA20 $\pm 2\sigma$ ，也可以根据实际情况进行调整，比如 MA10 $\pm 1.5\sigma$



Project：沪市指数预测（Prophet）



Facebook prophet的突变点分析



Prophet 的突变点机制能自动适应趋势变化，非常适合业务场景中突发事件的建模。

传统ARIMA模型，假设趋势是平滑变化的，无法灵活处理突发性趋势转折。

Prophet的解决方案：通过突变点检测，Prophet 将时间序列分段建模，每段的趋势斜率可以独立调整。例如：产品销量因营销活动突然增长。或者经济指标因政策调整骤降。

周期性分析

我想用prophet，对某支股票进行周期性分析，分析trend, weekly, yearly的情况，并通过prophet进行可视化，呈现给用户

编写prophet_analysis，股票周期性分析工具，这里应该也要先从远端阿里云MySQL中获取该股票的数据，然后默认检测过去1年的规律，用户也可以指定分析的时间范围。

在@stock_query_assistant-4.py 的基础上进行编写，写入到 stock_query_assistant-5.py

```
stock_query_assistant-5.py (new) +78 •  
from prophet import Prophet  
  
# exc_sql, arima_stock, boll_detection工具同前 ...  
  
@register_tool('prophet_analysis')  
class ProphetAnalysisTool(BaseTool):  
    description = '对指定股票(ts_code)的收盘价进行Prophet  
    parameters = [  
        {  
            'name': 'ts_code',  
            'type': 'string',  
            'description': '股票代码，必填',  
            'required': True
```

```
# prophet_analysis工具  
@register_tool('prophet_analysis')  
class ProphetAnalysisTool(BaseTool):  
    description = '对指定股票(ts_code)的收盘价进行Prophet周期性分析，分解trend、  
    parameters = [  
        {  
            'name': 'ts_code',  
            'type': 'string',  
            'description': '股票代码，必填',  
            'required': True  
        },  
        {  
            'name': 'start_date',  
            'type': 'string',  
            'description': '分析起始日期，格式YYYY-MM-DD，选填',  
            'required': False  
        },  
        {  
            'name': 'end_date',  
            'type': 'string',  
            'description': '分析结束日期，格式YYYY-MM-DD，选填',  
            'required': False  
        }  
    ]
```

周期性分析

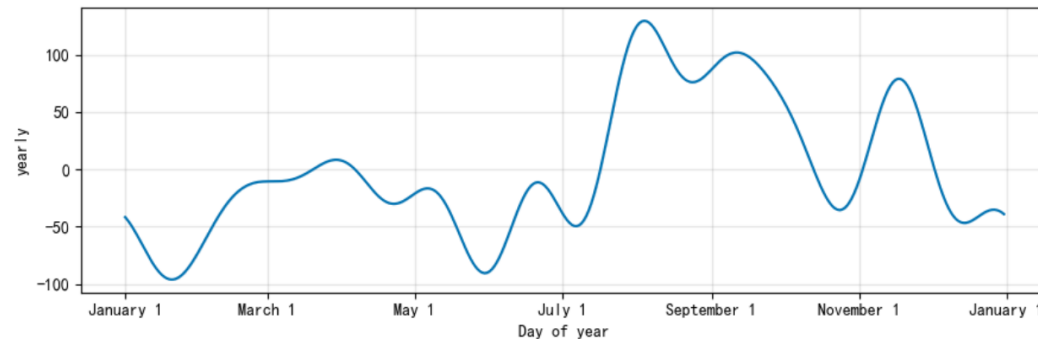
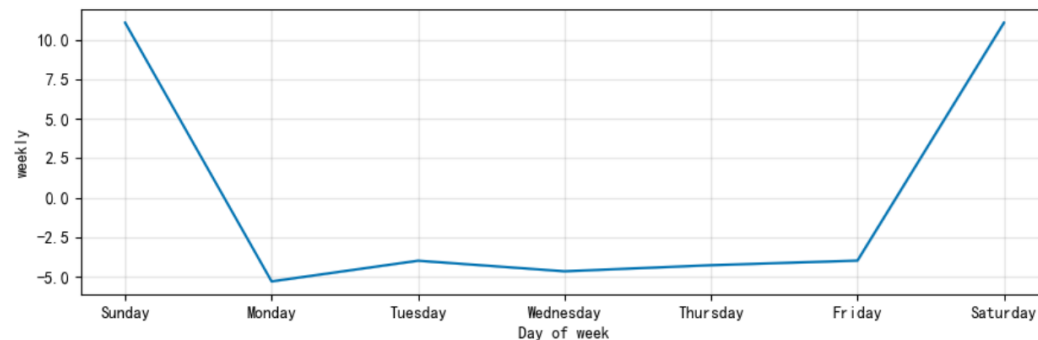
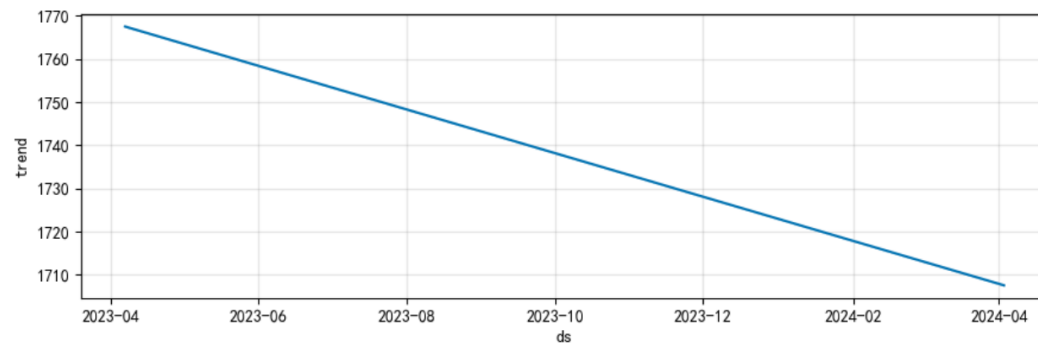
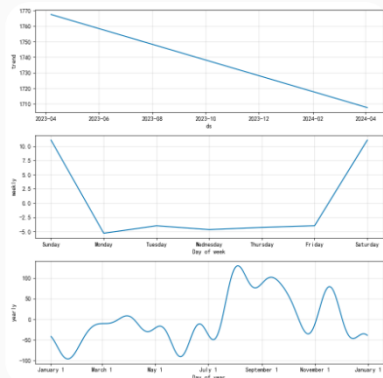
分析贵州茅台近一年周期性规律

▼ Start calling tool "prophet_analysis" ...

```
{"ts_code": "600519.SH", "start_date": "2023-04-07", "end_date": "2024-04-07"}
```

▼ Finished tool calling.

Prophet周期性分解 (趋势、周、年):



打卡：股票分析助手



开发完整的股票分析助手，提供：

- 特定股票的数据查询，可以基于某个时间段的范围
- 计算某个指标，比如MACD等，并绘制曲线
- 识别超买超卖点，比如基于Boll布林带
- 识别周期性的规律，比如使用prophet进行分析
- 对未来N天价格进行预测，比如使用ARIMA, prophet等
- 提供股票的新闻资讯，比如使用 tavily MCP
- 提供股票的基本面分析，比如提供 投研报告，作为files知识库
-

通过交互式BI，让大模型与专业模型进行深度结合。

大模型：与用户交互，理解用户的问题并回答

专业模型：提供建模，对未来进行预测，识别突变点等



Thank You
Using data to solve problems